

Design Load Balancer

Last Updated: December 29, 2025



Ashish Pratap Singh



High Priority

⌚ 44 min read



Listen to this chapter



Unlock Audio

🏗️ What is a Load Balancer?



A Load Balancer is a critical infrastructure component that distributes incoming network traffic across multiple servers to ensure no single server becomes overwhelmed.



Simulation "load-balancer" not found in registry

Simulation path: load-balancer

The core idea is straightforward: instead of routing all traffic to one server (which would eventually crash under heavy load), a load balancer acts as a traffic cop, intelligently spreading requests across a pool of healthy servers. This improves application availability, responsiveness, and overall system reliability.

Popular Examples: AWS Elastic Load Balancer (ELB), NGINX, HAProxy

Load balancers appear in almost every distributed system architecture. Understanding how to design one from scratch demonstrates deep knowledge of networking, high availability, and system scalability.

In this chapter, we will explore the **high-level design of a Load Balancer**.

Lets start by clarifying the requirements.

1. Clarifying Requirements

Before starting the design, it's important to ask thoughtful questions to uncover hidden assumptions, clarify ambiguities, and define the system's scope more precisely.

Here is an example of how a discussion between the candidate and the interviewer might unfold:

DISCUSSION

Candidate: "What type of traffic should the load balancer handle? Are we focusing on HTTP/HTTPS traffic, or should it support any TCP/UDP traffic?"

Interviewer: "Let's design a general-purpose load balancer that supports both Layer 4 (TCP/UDP) and Layer 7 (HTTP/HTTPS) load balancing."

Candidate: "What is the expected scale? How many requests per second should the system handle?"

Interviewer: "The load balancer should handle up to 1 million requests per second at peak traffic."

Candidate: "How should we handle server failures? Should the load balancer automatically detect and route around unhealthy servers?"

Interviewer: "Yes, health checking is critical. The system should detect failures within seconds and stop routing traffic to unhealthy servers."

Candidate: "Do we need to support session persistence, where requests from the same client go to the same backend server?"

Interviewer: "Yes, sticky sessions should be supported for stateful applications, but it should be configurable."

Candidate: "What about SSL/TLS termination? Should the load balancer handle encryption?"

Interviewer: "Yes, SSL termination at the load balancer is required to offload encryption work from backend servers."

Candidate: "What availability target should we aim for?"

Interviewer: "The load balancer itself must be highly available with 99.99% uptime, since it's on the critical path for all traffic."

After gathering the details, we can summarize the key system requirements.

1.1 Functional Requirements

- **Traffic Distribution:** Distribute incoming requests across

multiple backend servers using configurable algorithms.

- **Health Checking:** Continuously monitor backend servers and automatically remove unhealthy ones from the pool.
- **Session Persistence:** Support sticky sessions to route requests from the same client to the same server.
- **SSL Termination:** Handle SSL/TLS encryption and decryption to offload work from backend servers.
- **Layer 4 and Layer 7 Support:** Support both transport-level (TCP/UDP) and application-level (HTTP/HTTPS) load balancing.

1.2 Non-Functional Requirements

Beyond features, we need to consider the qualities that make the system production-ready:

- **High Availability:** The load balancer must be highly available (99.99% uptime) with no single point of failure.
- **Low Latency:** Should add minimal latency to requests (< 1ms overhead).
- **High Throughput:** Handle up to 1 million requests per second at peak.
- **Scalability:** Should scale horizontally to handle increasing traffic.
- **Fault Tolerance:** Continue operating even when individual components fail.

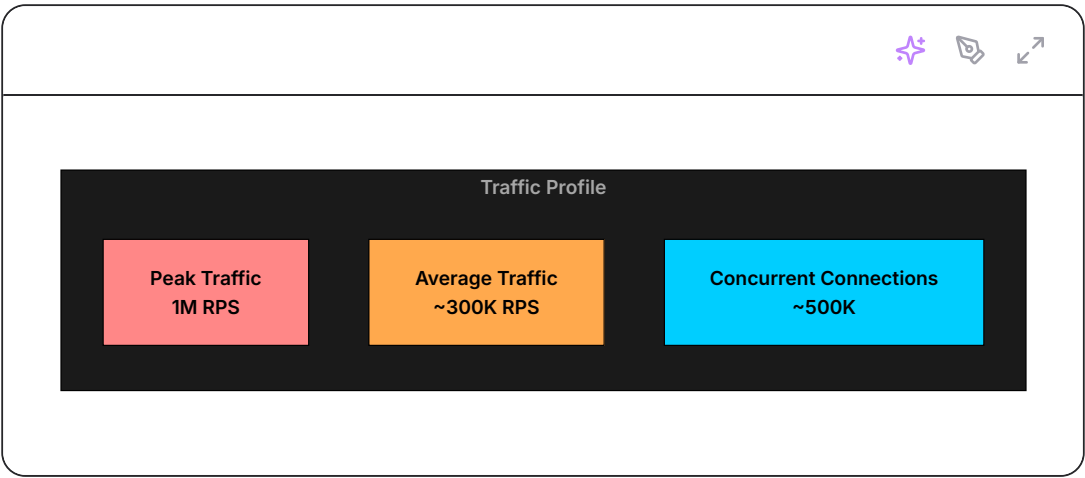
2. Back-of-the-Envelope Esti-

mation

Before diving into the design, let's understand the scale we are working with. These numbers will guide our architectural decisions, particularly around memory management and network capacity.

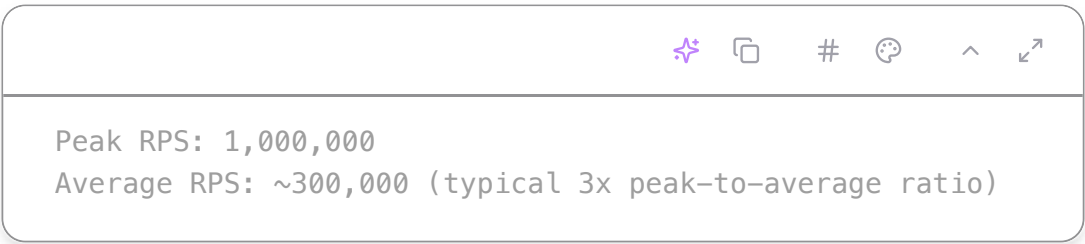
2.1 Traffic Estimates

Starting with the numbers from our requirements:



Request Volume

Our target is 1 million requests per second at peak. Traffic is rarely uniform, so let's assume the average is about one-third of peak:



Concurrent Connections

How many connections are open at any given time? This depends on connection duration. If the average request takes 500ms to complete (including network round trip and server processing):

```
Concurrent connections = RPS × average duration
                        = 1,000,000 × 0.5 seconds
                        = 500,000 connections
```

Half a million concurrent connections is substantial but manageable with modern hardware.

2.2 Bandwidth Estimates

Network bandwidth is often the first bottleneck at high throughput. Let's calculate what we need:

```
Average request size: ~2 KB (HTTP headers + small payload)
Average response size: ~10 KB (typical API response)
```

```
Ingress bandwidth = 1M RPS × 2 KB = 2 GB/s
Egress bandwidth  = 1M RPS × 10 KB = 10 GB/s
Total bandwidth   = 12 GB/s = ~96 Gbps
```

This is significant. A single 10 Gbps network card would not be enough. We need either multiple NICs, faster networking (25/40/100 Gbps), or multiple load balancer nodes to distribute the bandwidth requirement.

2.3 Health Check Overhead

Health checks generate their own traffic. With 1,000 backend servers and checks every 5 seconds:

```
Health check traffic = 1,000 servers / 5 seconds = 200 checks
```

This is negligible compared to user traffic. Health checking will not be a bottleneck.

2.4 Memory Requirements

Each active connection needs state tracking in memory:

Data	Size
Source IP + port	6 bytes
Destination IP + port	6 bytes
Connection state	4 bytes
Timestamps	16 bytes
Protocol-specific data	~200 bytes
Buffer space	~250 bytes
Total per connection	~500 bytes

For 500,000 concurrent connections:



Connection state memory = 500,000 × 500 bytes = 250 MB

250 MB is quite modest. Memory will not be our bottleneck. The bigger memory consideration is if we implement SSL termination, where session caches and certificate storage can consume several gigabytes.

2.5 Key Insights

These estimates tell us several important things about our design:

- Network bandwidth is the primary constraint.** At 12 GB/s, we need high-speed networking and likely multiple load balancer nodes.
- Memory is not a concern.** Connection tracking is cheap. Even 10x our estimate would only be 2.5 GB.
- Health checks are lightweight.** 200 checks per second is

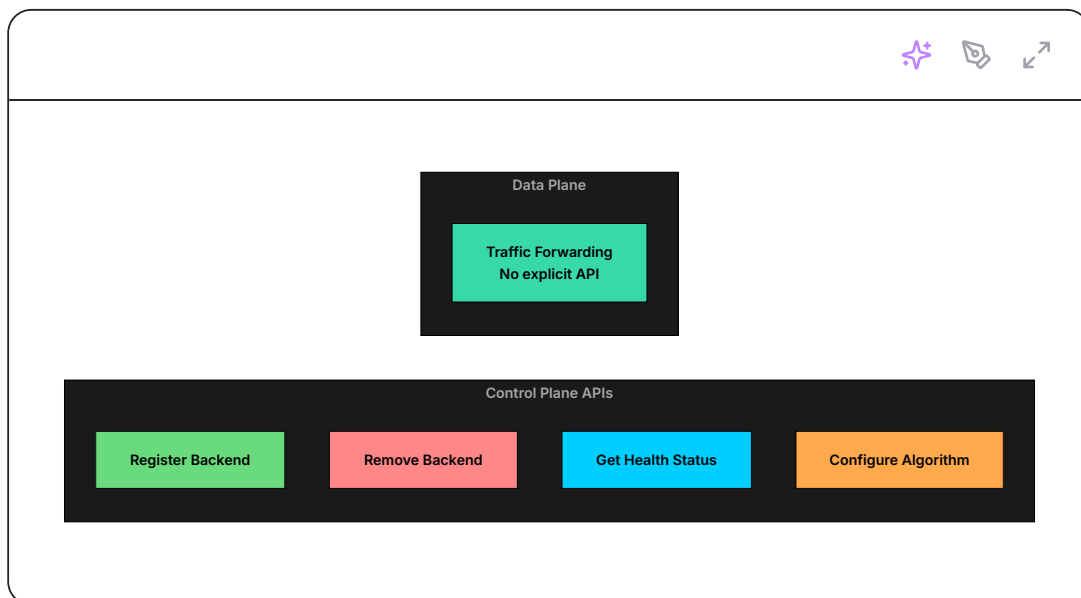
nothing compared to 1M requests per second.

4. **Horizontal scaling is necessary.** A single machine cannot handle 100 Gbps of traffic. We need to design for multiple load balancer nodes from the start.

3. Core APIs

A load balancer has two distinct interfaces. The **data plane** handles actual traffic (this is where millions of requests flow through), and the **control plane** handles configuration and management (registering backends, changing algorithms, viewing health status).

The data plane is implicit since it just forwards network traffic. The control plane needs explicit APIs for operators to manage the system. Let's define those.



3.1 Register Backend Server

Endpoint: **POST /backends**

When you spin up a new application server, you need to tell the load balancer about it. This endpoint adds a backend to the

pool and starts health checking it.

Request Body:

Parameter	Type	Re-quire d	Description
address	string	Yes	IP address or hostname of the back-end server
port	integer	Yes	Port the backend is listening on
weight	integer	No	Weight for weighted load balancing (default: 1)
health_check_path	string	No	HTTP path for health checks (default: /health)

Example Request:

{} Json

#

```
{
  "address": "10.0.1.15",
  "port": 8080,
  "weight": 2,
  "health_check_path": "/healthz"
}
```

Success Response (201 Created):

{} Json

#

```
{
  "backend_id": "backend-a1b2c3",
  "status": "unknown",
  "message": "Backend registered. Health check pending."
}
```

The initial status is "unknown" because we have not run a health check yet. Within a few seconds, it will transition to

"healthy" or "unhealthy."

Error Responses:

Status	Meaning
400 Bad Request	Invalid address format or port out of range
409 Conflict	A backend with this address:port already exists

3.2 Remove Backend Server

Endpoint: `DELETE /backends/{backend_id}`

When you want to decommission a server (for maintenance, scaling down, or replacement), this endpoint removes it from the pool. The load balancer will stop sending new traffic immediately, but existing connections are allowed to complete gracefully.

Path Parameter: `backend_id` - the ID returned when the backend was registered

Success Response (200 OK):

<code>{}</code> Json	     
<pre>{ "message": "Backend removed successfully", "backend_id": "backend-a1b2c3", "drained_connections": 42 }</pre>	

The `drained_connections` field tells you how many connections were in progress when the backend was removed.

Error Responses:

Status	Meaning
404 Not Found	No backend exists with this ID

3.3 Get Backend Health Status

Endpoint: `GET /backends/{backend_id}/health`

Returns detailed health information about a specific backend, useful for debugging and monitoring dashboards.

Success Response (200 OK):

{} Json

#

```
{
  "backend_id": "backend-a1b2c3",
  "status": "healthy",
  "last_check": "2024-01-15T10:30:00Z",
  "response_time_ms": 12,
  "active_connections": 847,
  "total_requests": 1250000,
  "failed_requests": 23
}
```

The response includes both current state (status, active connections) and historical metrics (total requests, failures) to help operators understand backend performance over time.

3.4 Configure Load Balancing Algorithm

Endpoint: `PUT /config/algorithm`

Changes how traffic is distributed across backends. This takes effect immediately for new connections (existing connections are not affected).

Request Body:

Parameter	Type	Required	Description
<code>algorithm</code>	string	Yes	One of: <code>round_robin</code> , <code>weighted_round_robin</code> , <code>least_connections</code> , <code>ip_hash</code> , <code>random</code>

<code>sticky_sessions</code>	boolean	Enable session persistence (default: false)
<code>sticky_ttl_seconds</code>	integer	How long sticky sessions last (default: 3600)

Example Request:

{} Json

☆
📄
#
🎨
^
↗

```

{
  "algorithm": "least_connections",
  "sticky_sessions": true,
  "sticky_ttl_seconds": 1800
}
```

Success Response (200 OK):

{} Json

☆
📄
#
🎨
^
↗

```

{
  "message": "Configuration updated",
  "algorithm": "least_connections",
  "sticky_sessions": true
}
```

Error Responses:

Status	Meaning
<code>400 Bad Request</code>	Unknown algorithm name or invalid TTL value

4. High-Level Design

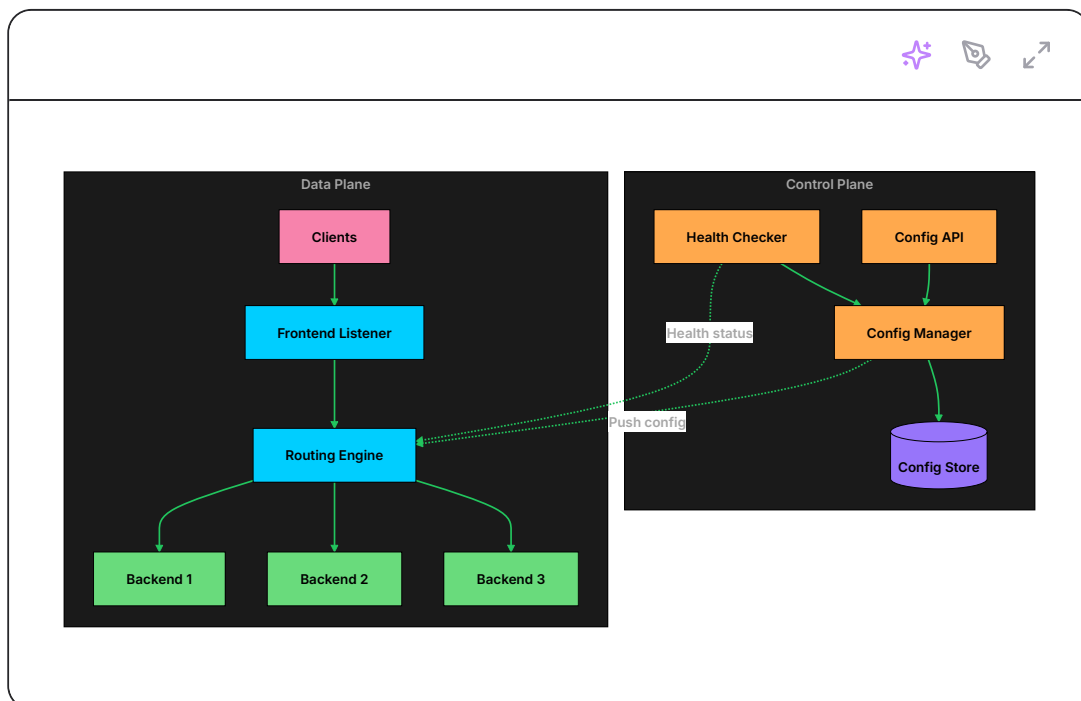
Now we get to the interesting part: designing the system architecture. Rather than presenting a complex diagram upfront, we

will build the design incrementally, starting with the simplest solution and adding components as we encounter challenges. This approach mirrors how you would tackle this problem in an interview.

Our load balancer needs to do three fundamental things:

1. **Distribute Traffic:** Accept incoming requests and forward them to healthy backend servers.
2. **Monitor Health:** Continuously check backends and route around failures.
3. **Stay Available:** The load balancer itself cannot be a single point of failure.

The architecture naturally splits into two parts: the **data plane** that handles actual traffic at high speed, and the **control plane** that manages configuration and health checking at a slower pace.



The data plane must be fast since every request flows through it. The control plane can be slower since configuration changes and health checks happen infrequently compared to request

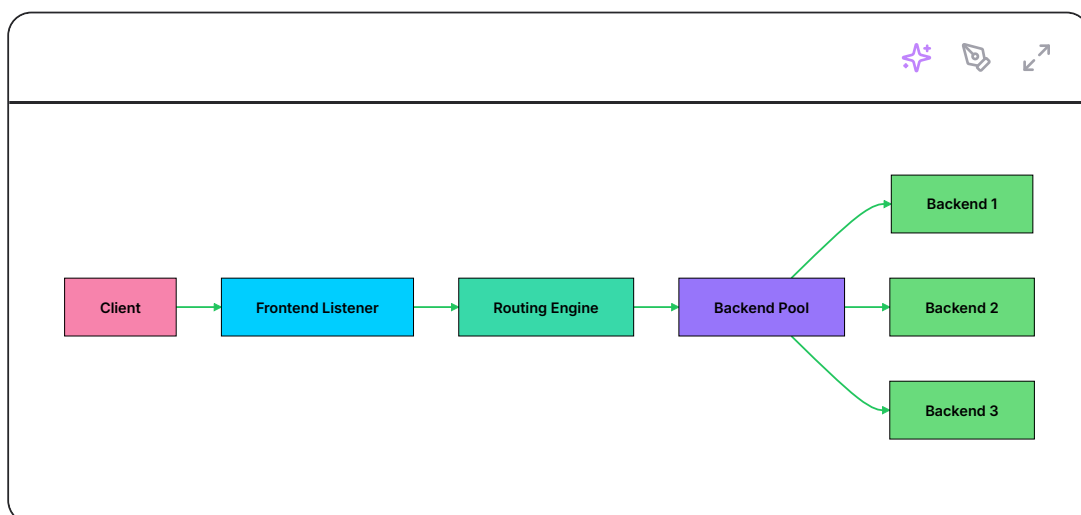
traffic.

Let's build this architecture step by step, starting with the most basic requirement: getting traffic from clients to backends.

4.1 Requirement 1: Traffic Distribution

The most basic job of a load balancer is accepting connections from clients and forwarding them to backend servers. This sounds simple, but there are several components involved, and understanding how they work together is key to designing a good system.

Components for Traffic Distribution



Let's introduce the components we need.

Frontend Listener

This is the entry point for all client traffic. The frontend listener binds to one or more ports (typically 80 for HTTP, 443 for HTTPS) and accepts incoming TCP connections.

When a connection arrives, the listener needs to:

- Accept the TCP connection from the client

- For Layer 7 load balancing, parse enough of the request to make routing decisions (HTTP headers, URL path, etc.)
- Hand off the connection to the routing engine for backend selection

In high-performance load balancers like NGINX or HAProxy, this component is optimized to handle hundreds of thousands of connections efficiently using techniques like `epoll` (on Linux) or `kqueue` (on BSD).

Routing Engine

This is the brain of the load balancer. Given a connection, the routing engine decides which backend server should handle it.

The routing engine maintains:

- A list of available backend servers and their metadata (address, port, weight)
- The current state of each backend (healthy, unhealthy, draining)
- Counters for connection tracking (needed for least-connections algorithm)
- The configured load balancing algorithm

When the frontend listener hands off a connection, the routing engine applies the configured algorithm (round robin, least connections, etc.) and returns the selected backend.

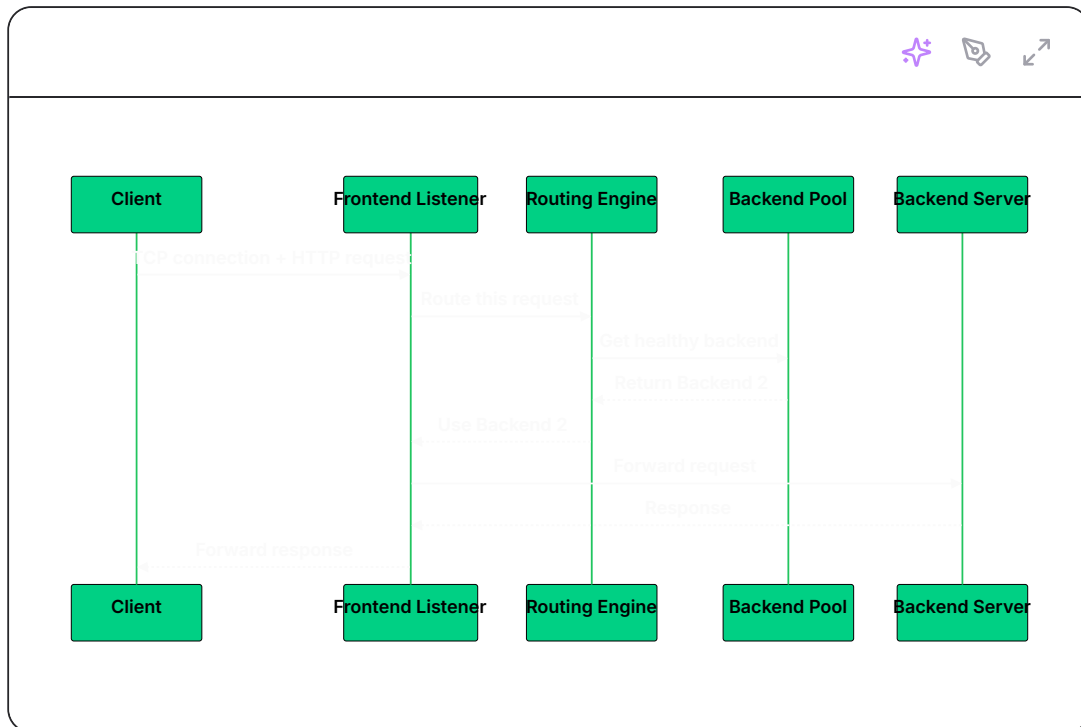
Backend Pool

A backend pool is a logical group of servers that can handle the same type of traffic. In simple setups, you might have just one pool. In more complex architectures, you might have separate pools for different services (one for API servers, another for static content, etc.).

Each pool contains:

- A list of backend servers with their addresses and ports
- Pool-specific configuration (algorithm, health check settings)
- Health status for each backend in the pool

How a Request Flows Through the System



Let's trace a request through the system:

1. **Connection arrives:** A client sends an HTTP request to the load balancer's public IP address (e.g., <https://api.example.com>).
2. **Frontend listener accepts:** The listener accepts the TCP connection and, for HTTP traffic, reads enough of the request to understand what is being asked (the URL, headers, etc.).
3. **Routing decision:** The listener asks the routing engine to select a backend. The engine checks the backend pool, filters out unhealthy servers, and applies the configured algo-

rithm.

4. **Forward to backend:** The request is forwarded to the selected backend server. This might be a simple proxy (read request, forward, read response) or a more sophisticated connection multiplexing setup.
5. **Return response:** The backend processes the request and sends a response. The load balancer forwards this back to the client.
6. **Connection handling:** Depending on configuration, the client connection might be kept alive for additional requests (HTTP keep-alive) or closed.

At this point, we have basic load balancing working. But what happens when Backend 2 crashes? Without health checking, the load balancer would keep sending traffic to it, and users would see errors. Let's address that next.

4.2 Requirement 2: Health Monitoring

Servers fail. It is not a matter of if, but when. A process might crash, a disk might fill up, or a network partition might isolate a server. Without health checking, the load balancer would blindly keep sending traffic to dead servers, and users would see errors.

Health monitoring solves this by continuously checking each backend and automatically routing around failures.

The Health Checker Component

We need a new component: the Health Checker. This is a background process that periodically probes each backend server to verify it is working correctly.

The health checker is responsible for:

- Sending periodic probes to each backend (typically every 5-10 seconds)
- Tracking the history of successes and failures
- Deciding when a backend should be marked unhealthy (usually after 2-3 consecutive failures)
- Deciding when an unhealthy backend can be marked healthy again (after 2-3 consecutive successes)
- Notifying the routing engine whenever a backend's status changes

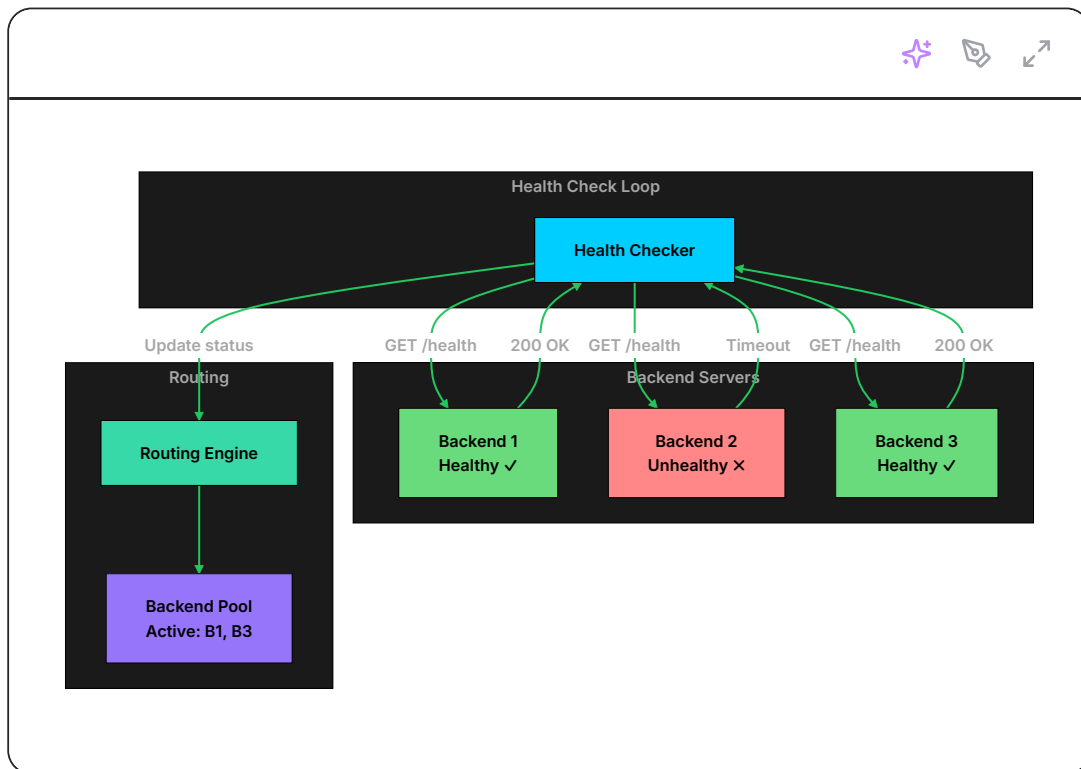
Types of Health Checks

Different applications need different types of health checks. A basic TCP check might be enough for some services, while others need a full HTTP request to verify the application is actually working.

Type	How It Works	When to Use
TCP Check	Opens a TCP connection and closes it	Basic connectivity verification. Fast but does not verify the application is working.
HTTP Check	Sends an HTTP request (GET /health), expects 2xx response	Web applications. Verifies the app can handle requests.
Custom Script	Runs a user-defined command or script	Complex checks like database connectivity, queue depth, etc.

Most production deployments use HTTP health checks. The application exposes a `/health` or `/healthz` endpoint that returns 200 OK when everything is working, and returns an error (or times out) when something is wrong.

How Health Checking Works



Here is what happens when a backend fails:

1. **Continuous monitoring:** The health checker sends probes to each backend every 5 seconds. Backends 1 and 3 respond with 200 OK. Backend 2 does not respond within the timeout.
2. **Failure tracking:** One failed check is not enough to mark a backend unhealthy (it could be a network blip). After 3 consecutive failures, Backend 2 is marked unhealthy.
3. **Status update:** The health checker notifies the routing engine that Backend 2 is now unhealthy.
4. **Traffic rerouting:** The routing engine immediately stops sending new traffic to Backend 2. Only Backends 1 and 3 receive requests.
5. **Recovery:** If Backend 2 starts responding again, it needs to pass several consecutive health checks before being marked healthy and returned to the pool. This prevents flapping (rapidly switching between healthy and unhealthy).

This entire process happens automatically. No human intervention is required to handle a failed server.

4.3 Requirement 3: High Availability

We have a problem. We built a load balancer to eliminate single points of failure in our backend, but the load balancer itself is now a single point of failure. If it crashes, all traffic stops. This is obviously unacceptable for a system targeting 99.99% uptime.

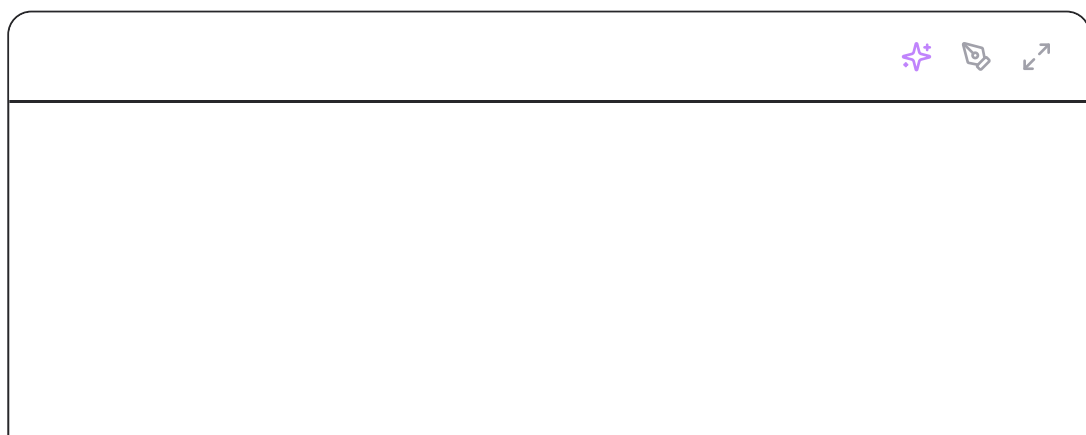
The solution is to run multiple load balancer instances. But this introduces new questions:

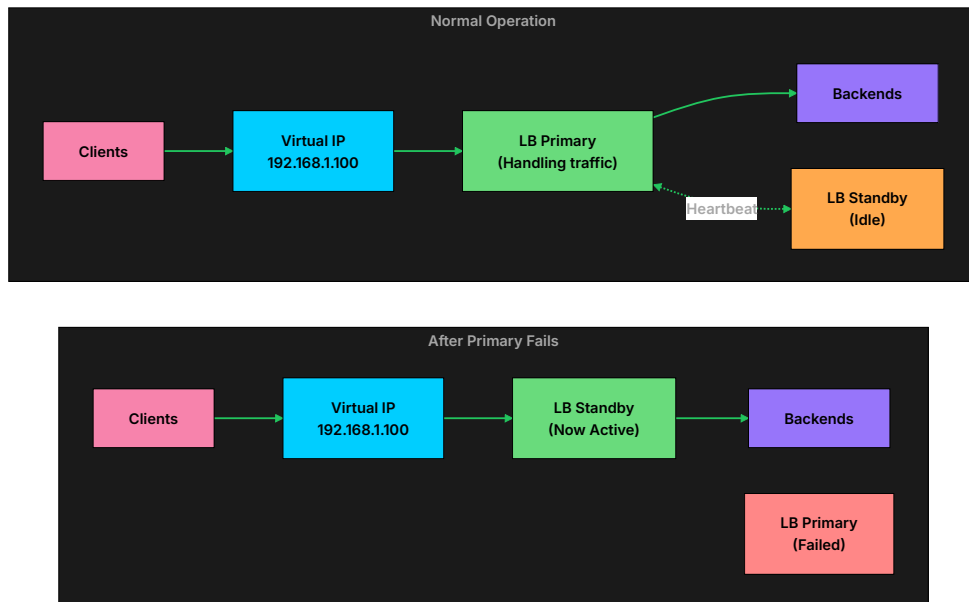
- How do clients know which instance to connect to?
- What happens when one instance fails?
- How do we handle state (like sticky sessions) across multiple instances?

Let's look at the two main patterns for achieving high availability.

Pattern 1: Active-Passive (Failover)

In this pattern, one load balancer handles all traffic while a backup waits idle, ready to take over if the primary fails.





How it works:

The key concept here is the Virtual IP (VIP). Clients connect to the VIP, not to either load balancer directly. Both load balancers know about this VIP, but only the active one actually responds to traffic on it.

The standby continuously monitors the primary using heartbeat messages (typically sent every second). If the primary stops responding, the standby "claims" the VIP by broadcasting an ARP message that says "I am now the owner of 192.168.1.100." Within 1-3 seconds, traffic starts flowing to the new active node.

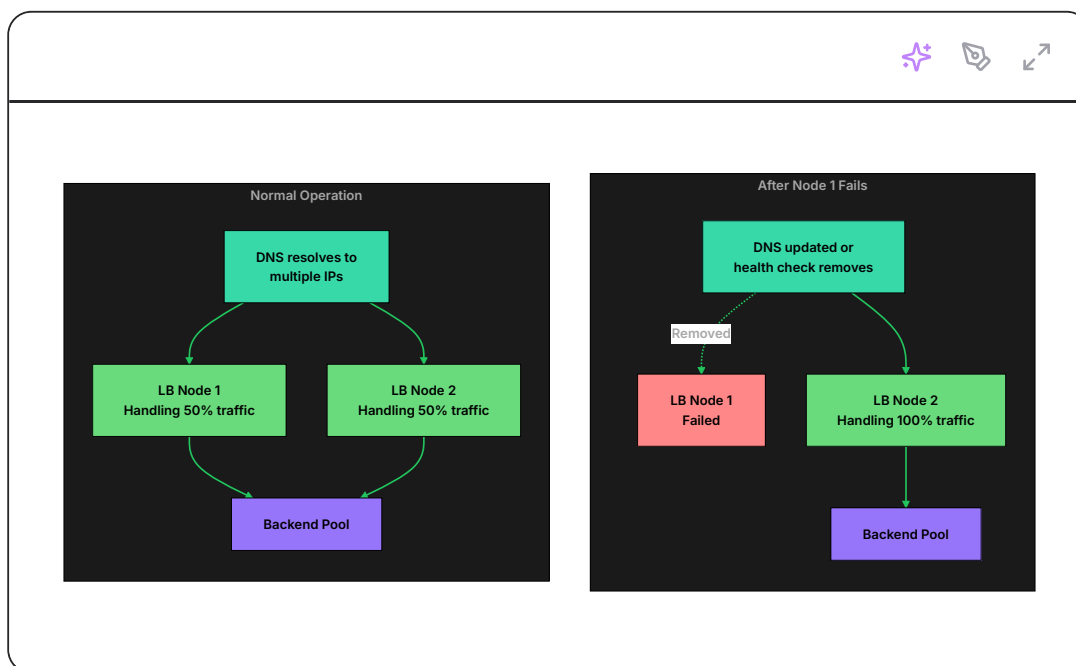
This pattern is commonly implemented using protocols like VRRP (Virtual Router Redundancy Protocol) or vendor-specific solutions like AWS's Elastic Load Balancer.

Pros: Simple to set up and reason about. No state synchronization needed since only one node handles traffic at a time.

Cons: Half your capacity sits idle during normal operation. You are paying for servers that are not doing useful work.

Pattern 2: Active-Active

In this pattern, multiple load balancer nodes handle traffic simultaneously. If one fails, the others continue without interruption.



How it works:

Traffic is distributed across multiple load balancer nodes using one of several methods:

1. **DNS round-robin:** DNS returns multiple IP addresses, and clients pick one. Not ideal since DNS caching can cause uneven distribution and slow failover.
2. **Upstream load balancer:** Another layer of load balancing (like a hardware L4 load balancer or cloud provider's network load balancer) distributes traffic to your L7 load balancers.
3. **Anycast:** All nodes share the same IP address. The network routes each connection to the nearest healthy node. This is how Cloudflare and other CDNs work at global scale.

Pros: All resources are utilized during normal operation. Better

throughput since traffic is spread across multiple nodes. More resilient since the failure of one node only reduces capacity rather than causing an outage.

Cons: More complex to manage. If you use sticky sessions, you need to synchronize session state across nodes (typically using a shared Redis cluster).

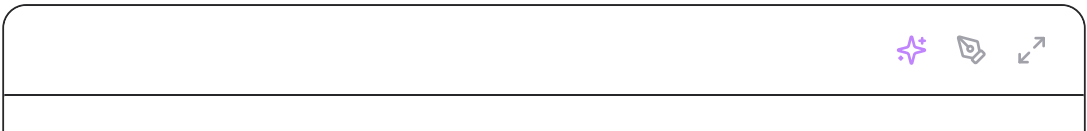
Which Pattern Should You Choose?

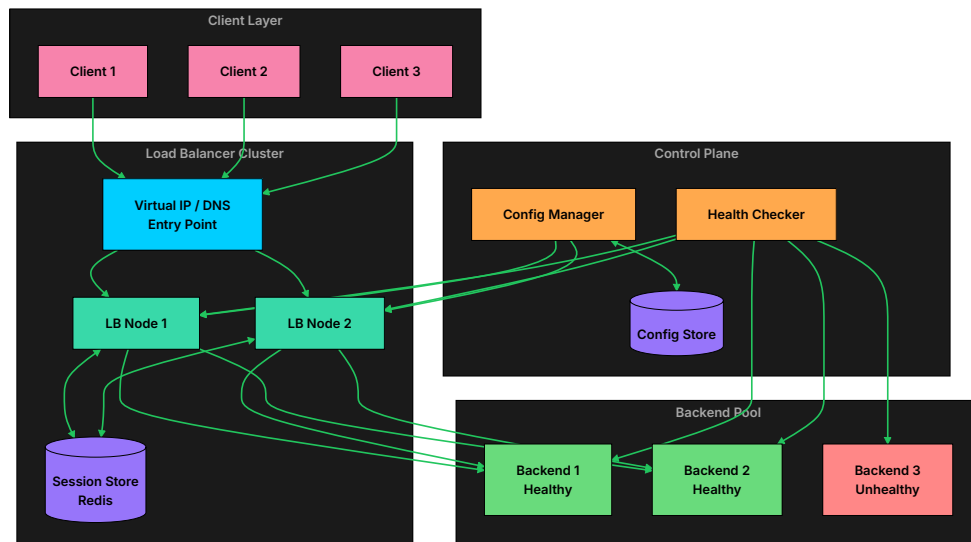
Consideration	Active-Passive	Active-Active
Complexity	Simpler	More complex
Resource utilization	50% (standby is idle)	100%
Failover time	1-3 seconds	Near-instant (traffic shifts to remaining nodes)
Sticky sessions	Easy (all state on one node)	Requires shared state store
Scale	Limited to one node's capacity	Scales horizontally

For most production systems handling significant traffic, **active-active** is the better choice despite its complexity. The benefits of full resource utilization and instant failover outweigh the added complexity of state synchronization.

4.4 Putting It All Together

Now that we have designed each piece, let's step back and see how they all fit together. Here is the complete architecture that satisfies our requirements for traffic distribution, health monitoring, and high availability.





Let's trace how everything works together:

1. **Clients connect to the VIP.** They do not know (or care) about the individual load balancer nodes. The VIP is a stable entry point that abstracts away the LB cluster.
2. **Traffic is distributed across LB nodes.** In active-active mode, both LB Node 1 and LB Node 2 handle traffic simultaneously.
3. **Each LB node routes to healthy backends.** Backend 3 failed health checks and is marked unhealthy (red), so no traffic goes to it. LB nodes only send traffic to Backends 1 and 2.
4. **Session state is shared.** If a user's first request hits LB Node 1 and sticky sessions are enabled, their session mapping is stored in Redis. If their next request happens to hit LB Node 2, it can still route them to the correct backend by looking up their session in Redis.
5. **Health checking runs continuously.** The Health Checker monitors all backends and both LB nodes. When it detects Backend 3 is unhealthy, it notifies both LB nodes to stop routing traffic there.
6. **Configuration is managed centrally.** The Config Manager

stores configuration (backend lists, algorithms, health check settings) in the Config Store and pushes updates to all LB nodes.

Component Summary

Component	What It Does	Key Characteristics
Virtual IP / DNS	Provides a stable entry point for clients	Abstracts away the LB cluster
LB Nodes	Accept connections and route to backends	Stateless, horizontally scalable
Session Store (Redis)	Stores sticky session mappings	Shared across all LB nodes
Health Checker	Monitors backend and LB health	Runs continuously in the background
Config Manager	Manages load balancer configuration	Handles API requests, persists config
Config Store	Persists configuration	Could be etcd, Consul, or a database
Backend Pool	The application servers being load balanced	Grouped by service type

This architecture handles millions of requests per second while automatically recovering from failures. The separation of data plane (LB nodes) and control plane (health checker, config manager) keeps the traffic path fast and simple.

5. Database Design

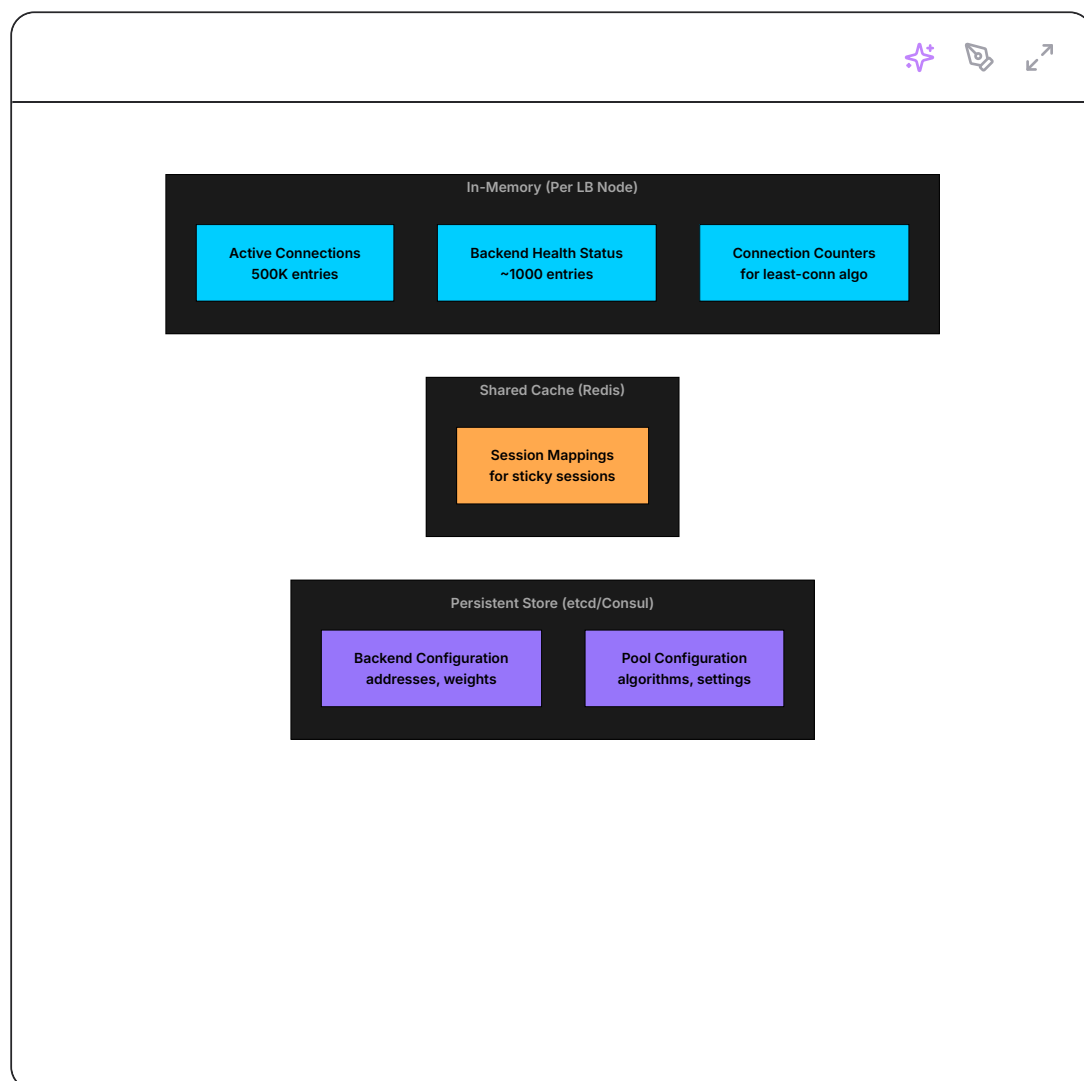
Here is something that might surprise you: a load balancer does not really need a traditional database for its core operation. The data plane (traffic forwarding) is entirely in-memory since every nanosecond counts when you are handling millions of requests per second.

However, the control plane does need persistent storage. Configuration (which backends exist, what algorithm to use, health check settings) needs to survive restarts. And if you are using sticky sessions in an active-active setup, session mappings need to be shared across LB nodes.

Let's think through what data lives where.

5.1 Where Does Data Live?

Different types of data have different requirements for speed, persistence, and sharing. Here is how we split things up:



Data Type	Storage Location	Why?
Active connections	In-memory on each LB node	Must be microsecond-fast. Each LB node manages its own connections.

Backend health status	In-memory on each LB node	Updated frequently (every health check). Pushed from health checker.
Connection counters	In-memory on each LB node	Updated on every request for least-connections algorithm.
Session mappings	Redis cluster	Needs to be shared across LB nodes for active-active sticky sessions.
Backend configuration	etcd/Consul/PostgreSQL	Persistent, versioned. Survives restarts and deployments.
Metrics and logs	Prometheus/InfluxDB	Historical data for dashboards and alerting.

The key insight is that the hot path (handling requests) should never touch disk or cross a network boundary for core routing decisions. Configuration is loaded into memory at startup and updated via push notifications when it changes.

5.2 Configuration Schema

The control plane needs to store information about backends and pools. Here is a schema that supports our API and features.

Backend Servers

Each backend server record contains everything the load balancer needs to route traffic to it:

Field	Type	Description
<code>backend_id</code>	String (PK)	Unique identifier (e.g., "backend-a1b2c3")
<code>pool_id</code>	String (FK)	Which pool this server belongs to
<code>address</code>	String	IP address or hostname
<code>port</code>	Integer	Port the server listens on
<code>weight</code>	Integer	Weight for weighted algorithms (default: 1)
<code>max_connec-</code>		

<code>tions</code>	Integer	Maximum concurrent connections (0 = unlimited)
<code>enabled</code>	Boolean	Administrative toggle to disable without removing
<code>created_at</code>	Time-stamp	When this backend was registered
<code>updated_at</code>	Time-stamp	Last configuration change

Backend Pools

Pools group related backends and define shared behavior:

Field	Type	Description
<code>pool_id</code>	String (PK)	Unique identifier (e.g., "pool-api-servers")
<code>name</code>	String	Human-readable name
<code>algorithm</code>	Enum	round_robin, weighted_round_robin, least_connections, ip_hash
<code>health_check_type</code>	Enum	tcp, http, custom
<code>health_check_path</code>	String	HTTP path for health checks (e.g., "/health")
<code>health_check_interval</code>	Integer	Seconds between health checks
<code>healthy_threshold</code>	Integer	Consecutive successes to mark healthy
<code>unhealthy_threshold</code>	Integer	Consecutive failures to mark unhealthy
<code>sticky_sessions</code>	Boolean	Enable session persistence
<code>sticky_ttl</code>	Integer	How long sticky sessions last (seconds)

5.3 Session Store Schema (Redis)

When sticky sessions are enabled, we need to remember

which backend each user should go to. Redis is perfect for this since it is fast, supports TTLs, and can be shared across LB nodes.

```
Key:    sticky:{client_identifier}  
Value:  backend-a1b2c3  
TTL:    1800 seconds (30 minutes, configurable)
```

The `client_identifier` depends on the stickiness method:

- For cookie-based stickiness: a session ID from the cookie
- For IP-based stickiness: the client's IP address
- For header-based stickiness: a hash of the relevant header value

When a request arrives:

1. Look up `sticky:{client_identifier}` in Redis
2. If found and the backend is healthy, route to that backend
3. If not found (or backend is unhealthy), apply normal algorithm and store the mapping

6. Design Deep Dive

The high-level architecture gives us a solid foundation, but system design interviews often go deeper. Interviewers want to see that you understand the trade-offs involved in key decisions.

In this section, we will explore the most important design choices in detail: load balancing algorithms, health checking strategies, session persistence, Layer 4 vs Layer 7, SSL termi-

nation, and handling load balancer failures.

6.1 Load Balancing Algorithms

The load balancing algorithm determines how traffic gets distributed across backends. This is one of the most important decisions in your design since it directly affects how evenly load is spread, how well the system handles failures, and how it behaves under various traffic patterns.

A good algorithm should:

- Distribute load reasonably evenly across healthy backends
- Avoid overloading any single server
- Work well with your specific traffic patterns (stateless vs stateful, uniform vs variable request costs)
- Be simple enough to execute quickly (remember, this runs on every request)

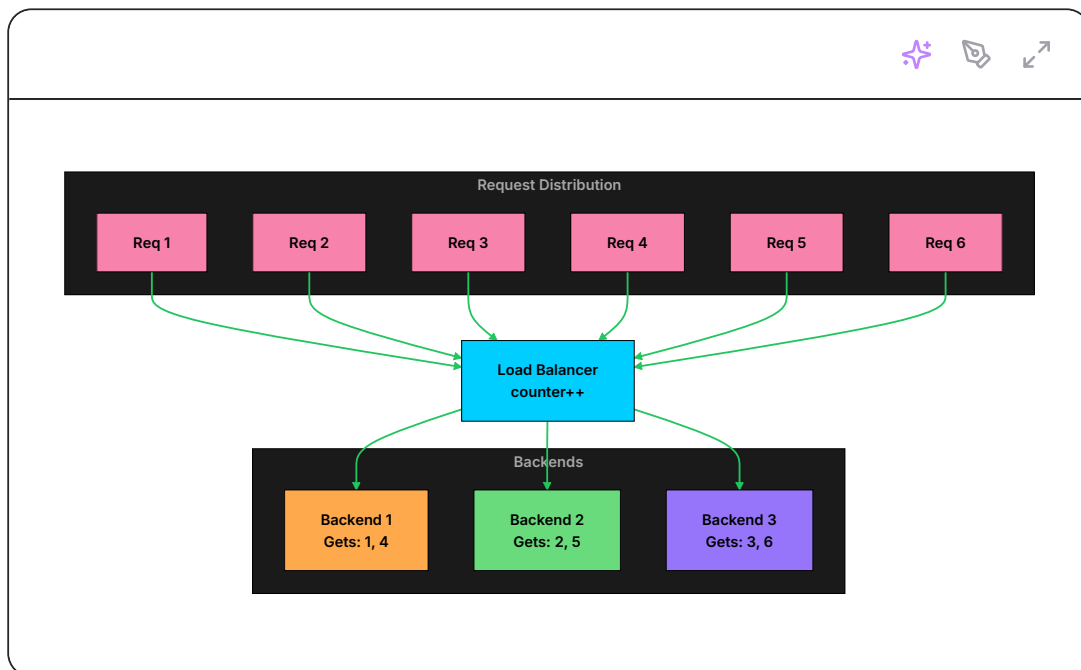
Let's explore the main options, starting with the simplest and building to more sophisticated approaches.

Approach 1: Round Robin

Round robin is the simplest load balancing algorithm, and often the default choice. Requests are distributed sequentially across backends: first request goes to Backend 1, second to Backend 2, third to Backend 3, then back to Backend 1.

How It Works

The algorithm maintains a counter that increments with each request. The selected backend is $\text{counter \% num_ber_of_backends}$.



The implementation is trivial:

```
counter = counter + 1
selected_backend = backends[counter % len(backends)]
```

Why use it?

Round robin's simplicity is its strength. There is no state to track beyond a single counter. Each LB node can run independently with its own counter since exact coordination is not necessary for good distribution over time. The algorithm adds essentially zero latency.

When it falls short

The simplicity comes with blind spots. Round robin treats all backends and all requests as equal, which is often not true:

- If Backend 1 is a powerful machine and Backend 2 is weaker, both get the same traffic, potentially overloading Backend 2.
- If some requests are expensive (complex queries, large file uploads), round robin does not account for this. A backend might get three expensive requests in a row while another

gets three cheap ones.

- If a backend is slow (due to garbage collection, disk I/O, etc.), round robin keeps sending traffic to it.

Best for

Homogeneous backends (same hardware, same configuration) handling uniform requests (similar processing cost per request). This is common for stateless API servers.

Approach 2: Weighted Round Robin

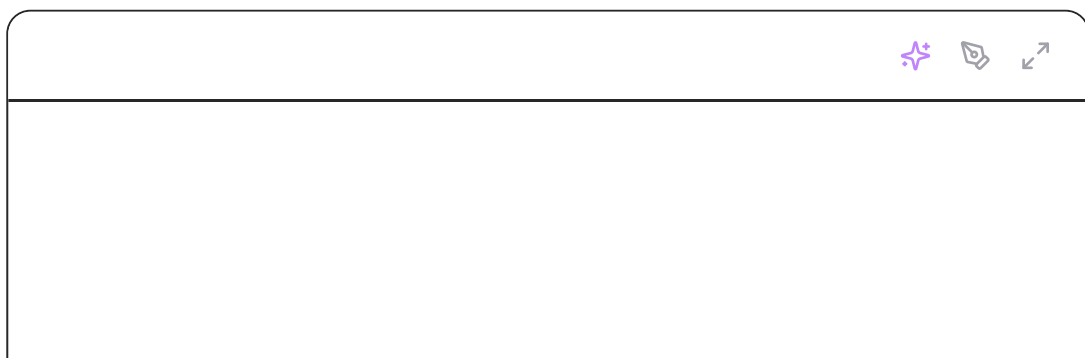
What if your backends are not identical? Maybe Backend 1 is a beefy 32-core machine while Backend 2 is a smaller 8-core instance. Sending equal traffic to both would waste capacity on Backend 1 and overload Backend 2.

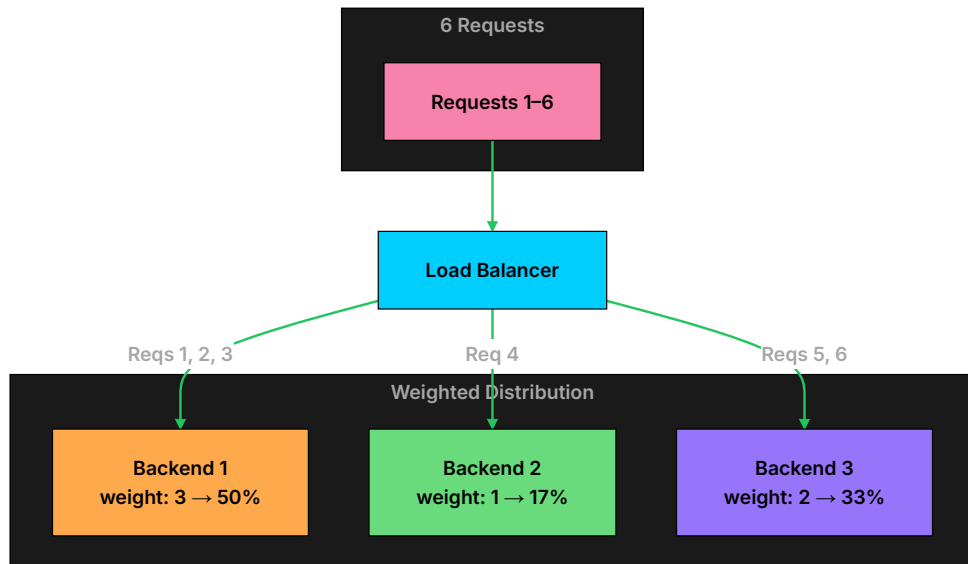
Weighted round robin solves this by assigning a weight to each backend. Backends with higher weights receive proportionally more traffic.

How It Works

If Backend 1 has weight 3, Backend 2 has weight 1, and Backend 3 has weight 2, the total weight is 6. Over every 6 requests:

- Backend 1 gets 3 requests (50%)
- Backend 2 gets 1 request (17%)
- Backend 3 gets 2 requests (33%)





Implementation considerations

A naive implementation might send the first 3 requests to Backend 1, then 1 to Backend 2, then 2 to Backend 3. This creates "bursts" to high-weight servers, which can cause momentary overload.

Better implementations like NGINX's "smooth weighted round robin" interleave requests more evenly. Instead of [1,1,1,2,3,3], you get something like [1,3,1,2,1,3], spreading the load more smoothly over time.

When to use it

Weighted round robin is great when you know your backends have different capacities and want to utilize them proportionally. Common scenarios:

- Mixed instance types (some large, some small)
- Gradual rollouts (new version gets weight 1, old version gets weight 9)
- Geographic distribution (nearby datacenter gets higher weight)

Limitations

The weights are static. If Backend 1 suddenly slows down due to a noisy neighbor or garbage collection, it keeps getting the same traffic. The algorithm does not adapt to runtime conditions.

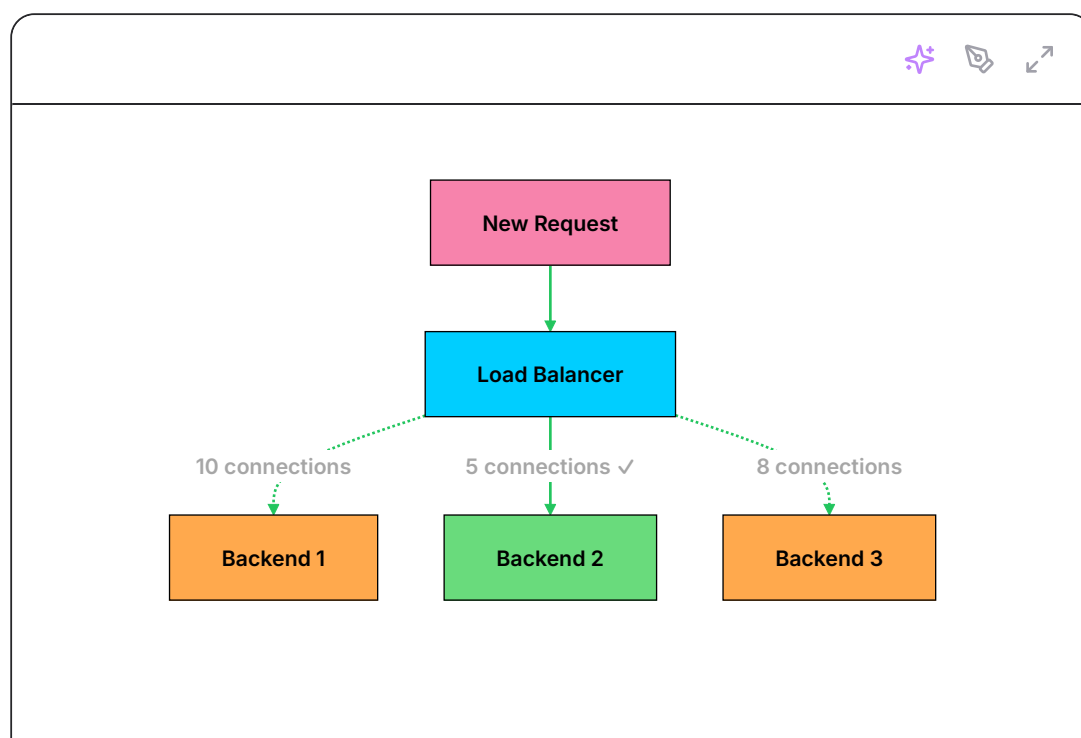
Approach 3: Least Connections

Both round robin and weighted round robin are "blind" to what is actually happening on your backends. They distribute requests based on a predetermined pattern, regardless of whether a backend is struggling with a heavy workload.

Least connections takes a different approach: it sends each new request to whichever backend currently has the fewest active connections.

How It Works

The load balancer tracks how many connections are currently active on each backend. When a new request arrives, it scans the list and picks the backend with the lowest count.



In this example, Backend 2 has only 5 active connections compared to 10 and 8 on the others. The new request goes to Backend 2.

Why this works well

Least connections naturally adapts to runtime conditions. If one backend is processing slow requests (maybe a complex database query or a large file upload), it accumulates connections and gets fewer new ones. Meanwhile, a backend that is processing requests quickly keeps its connection count low and picks up more traffic.

This self-balancing behavior is powerful. It handles:

- Backends with different capacities (faster servers complete requests faster, so they have fewer connections and get more traffic)
- Variable request costs (expensive requests tie up connections longer)
- Transient slowdowns (a backend doing garbage collection temporarily gets fewer requests)

The catch

Least connections requires state. Each LB node needs to track connection counts for every backend. In a distributed setup with multiple LB nodes, this gets tricky since each node only knows about its own connections. You have a few options:

- Each node tracks only its own connections (good enough in practice for most workloads)
- Nodes share connection counts via a shared cache (more accurate but adds latency)
- Use a hybrid approach where nodes periodically sync

counts

Best for

Workloads with variable request processing times, like applications that mix quick API calls with slow database queries or file operations.

Approach 4: Weighted Least Connections

This combines the best of weighted round robin and least connections. It accounts for both configured capacity (weights) and real-time load (connection counts).

How It Works

Instead of picking the backend with the fewest connections, we pick the one with the lowest ratio of connections to weight. A powerful server (high weight) can handle more connections before its ratio becomes high.

Backend 1: 10 connections, weight 5 → ratio = $10/5 = 2.0$
Backend 2: 6 connections, weight 2 → ratio = $6/2 = 3.0$
Backend 3: 4 connections, weight 2 → ratio = $4/2 = 2.0$

→ Backend 1 or 3 selected (tie at 2.0, pick first)

Even though Backend 1 has more connections (10) than Backend 3 (4), it has a higher weight, so its ratio is the same. Both are equally loaded relative to their capacity.

Why this is often the best choice

Weighted least connections gives you:

- Capacity-aware routing (like weighted round robin)

- Adaptive behavior (like least connections)
- Proportional load distribution (each backend stays at roughly the same utilization percentage)

This is why many production load balancers default to weighted least connections for HTTP traffic.

Complexity trade-off

The algorithm is more complex than round robin. You need to track connections per backend and calculate ratios on each request. But the computation is trivial (a few integer operations), and the better load distribution is usually worth it.

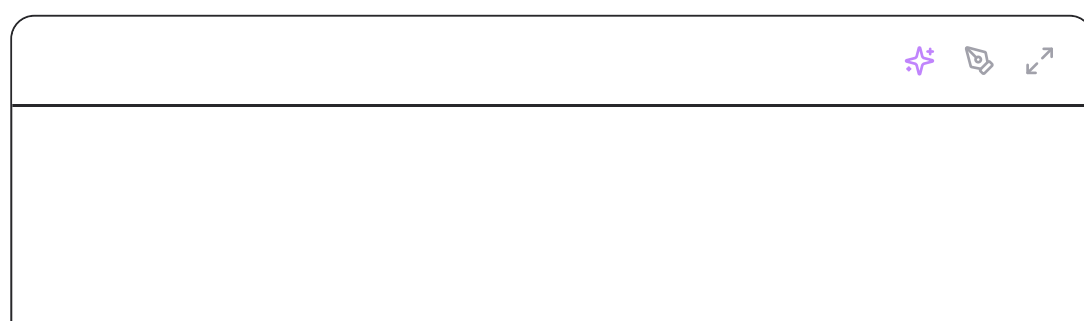
Approach 5: IP Hash (Source Hashing)

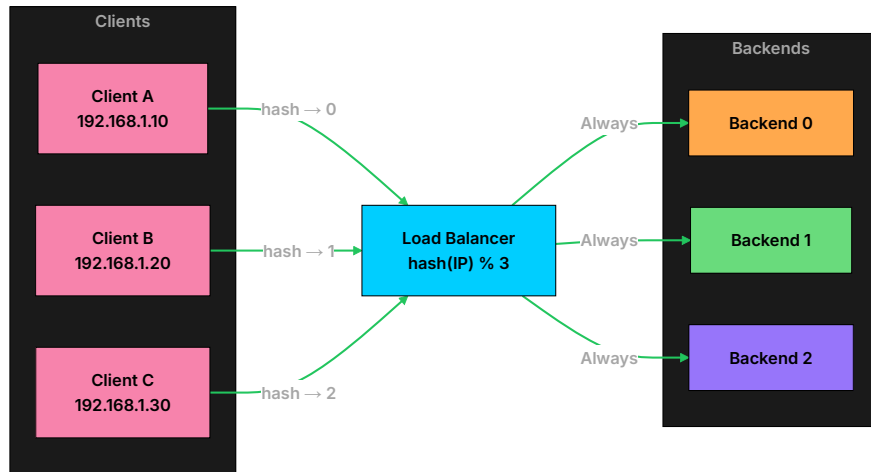
Sometimes you need session persistence, meaning requests from the same client should always go to the same backend. The simplest way to achieve this without storing any state is to use the client's IP address as a hash key.

How It Works

```
backend_index = hash(client_ip) % number_of_backends
```

The same IP address always produces the same hash, which always maps to the same backend (as long as the backend count does not change).





Client A (192.168.1.10) always routes to Backend 0. Client B always routes to Backend 1. No state needs to be stored anywhere.

Why use it

IP hash gives you session persistence at Layer 4, without needing to parse HTTP or store session mappings. Every LB node can compute the same hash independently since the algorithm is deterministic.

This is useful when:

- You cannot modify the application to use cookies
- You need stickiness for non-HTTP protocols
- You want to avoid the complexity of a shared session store

Problems with IP hash

The approach has significant limitations:

1. **NAT breaks it.** If thousands of users are behind a corporate

NAT, they all share the same public IP. All of them will hit the same backend, creating a hot spot.

2. **Distribution can be uneven.** Hash functions try to spread values evenly, but with a small number of backends, you can get unlucky distributions where some backends get more traffic than others.
3. **Adding/removing backends causes massive reshuffling.** If you go from 3 backends to 4, the formula changes from `hash % 3` to `hash % 4`. Most clients will map to different backends, losing their sessions.

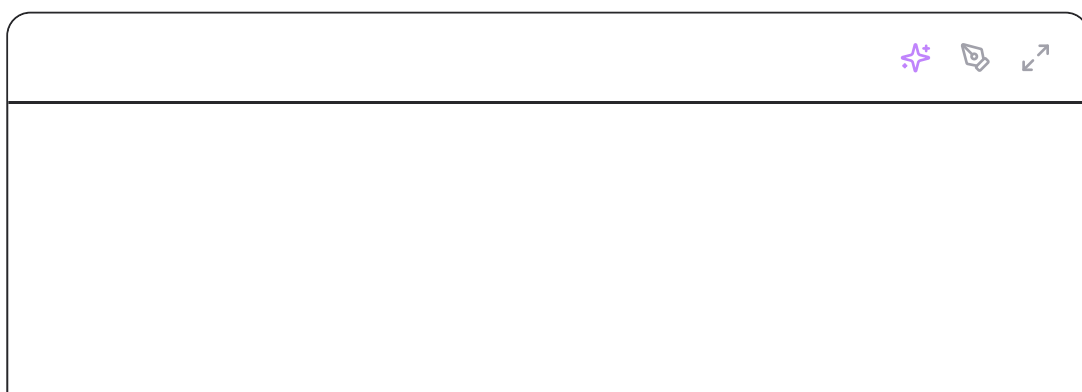
The last problem is serious enough that consistent hashing was invented specifically to solve it.

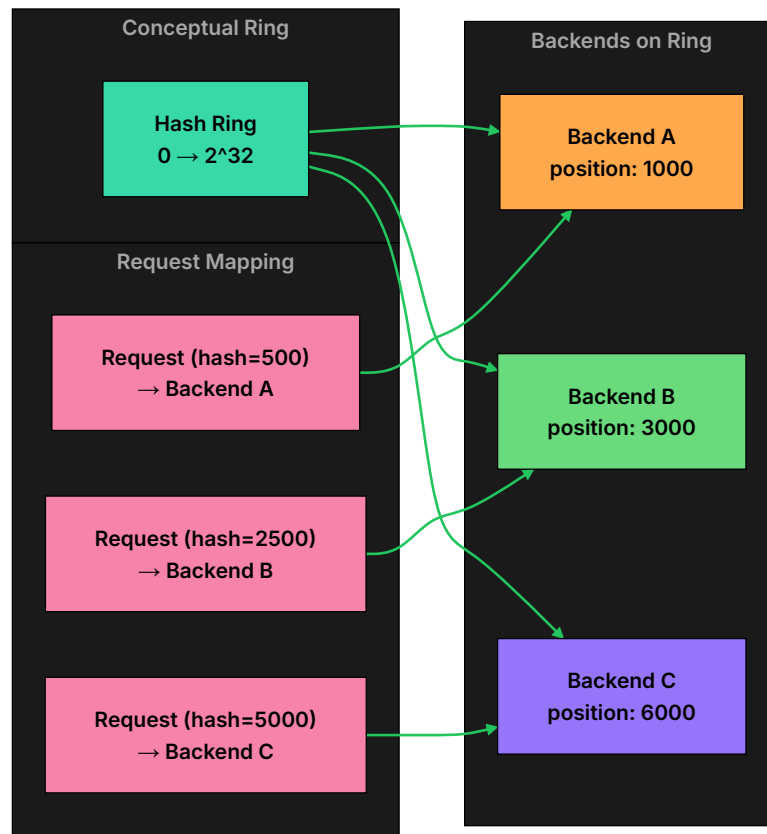
Approach 6: Consistent Hashing

Consistent hashing solves the "massive reshuffling" problem of regular hashing. When you add or remove a backend, only a small fraction of requests get remapped instead of most of them.

How It Works

Imagine a ring numbered from 0 to 2^{32} . Both backends and requests get hashed onto this ring. Each request goes to the first backend found when walking clockwise from its hash position.





Why this is better than simple hashing

Consider what happens when Backend B is removed:

- Requests that were going to Backend A still go to Backend A (no change)
- Requests that were going to Backend C still go to Backend C (no change)
- Only requests that were going to Backend B get remapped (they now go to Backend C)

With simple `hash % n` hashing, changing from 3 backends to 2 would remap roughly 2/3 of all requests. With consistent hashing, only 1/3 get remapped (the requests that were going

to the removed backend).

Virtual nodes for better distribution

One problem with basic consistent hashing is that backends might be unevenly distributed on the ring. Backend A might end up responsible for a huge arc while Backend C gets a tiny one.

The solution is virtual nodes: instead of placing each backend once on the ring, place it multiple times (e.g., 100-200 positions per backend). This evens out the distribution significantly.

When to use consistent hashing

Consistent hashing shines in scenarios with dynamic backend pools:

- Auto-scaling groups that add/remove servers based on load
- Cache servers where remapping causes cache misses
- Distributed databases where remapping requires data migration

For most load balancing use cases where backends are relatively stable, the added complexity is not worth it. But if you are building a distributed cache or a system with frequent scaling events, consistent hashing is valuable.

Choosing an Algorithm

Here is how the algorithms compare across key dimensions:

Algorithm	Complexity	Statefulness	Adaptiveness	Best For
Round Robin	Very simple	Stateless	None	Homogeneous backends, uniform requests

Weighted Round Robin	Simple	Stateless	None	Known capacity differences
Least Connections	Moderate	Per-node state	High	Variable request processing times
Weighted Least Conn	Moderate	Per-node state	High	Production with mixed servers
IP Hash	Simple	Stateless	None	Basic session persistence
Consistent Hash	Complex	Stateless	None	Dynamic scaling, caching

Recommendations:

For most HTTP applications, start with **Weighted Least Connections**. It handles heterogeneous backends and variable request costs gracefully, adapting to runtime conditions automatically.

If your backends are identical and your requests are uniform (a rare but possible scenario), **Round Robin** is fine and slightly faster.

For session persistence, prefer **cookie-based stickiness** (covered in section 6.3) over IP hash when possible. IP hash is fragile due to NAT and uneven distribution.

Use **Consistent Hashing** when backends frequently scale up/down, or when you are load balancing cache servers where cache locality matters.

6.2 Health Checking Strategies

We touched on health checking in the high-level design, but there is more depth to explore.

The health checking strategy you choose affects how quickly you detect failures, how aggressively you mark servers unhealthy, and how smoothly you handle recovery.

Configuring Health Checks

Every health check has several configurable parameters. Getting these right is important since too aggressive and you will get false positives (marking healthy servers as unhealthy), too passive and you will keep routing to failed servers longer than necessary.

Parameter	What It Controls	Typical Value	Trade-off
Interval	Time between checks	5-10 seconds	Lower = faster detection, higher load
Timeout	Max wait for response	2-3 seconds	Lower = faster detection, more false positives
Healthy Threshold	Passes needed to mark healthy	2-3	Higher = more stable, slower recovery
Unhealthy Threshold	Failures needed to mark unhealthy	2-3	Higher = more stable, slower failure detection

A common configuration is: check every 5 seconds, timeout after 3 seconds, require 3 consecutive failures to mark unhealthy, and require 2 consecutive successes to mark healthy again.

Types of Health Checks

TCP Health Check

The simplest form. The load balancer opens a TCP connection to the backend and immediately closes it. If the connection succeeds, the backend is considered healthy.

1. Open TCP connection to backend:8080
2. Connection succeeds within timeout → HEALTHY
3. Connection refused or timeout → UNHEALTHY

This verifies network connectivity and that something is listening on the port, but it does not verify the application is actually working. A process could be hung, accepting connections but never responding to requests.

HTTP Health Check

The load balancer sends an HTTP request and checks the response. This is the standard for web applications.

1. Send: GET /health HTTP/1.1
2. Expect: 200 OK (within timeout)
3. Any other response or timeout → UNHEALTHY

The `/health` endpoint should do meaningful checks. A good health endpoint might:

- Verify database connectivity
- Check that required services are reachable
- Return 200 only if the service can actually handle requests

A bad health endpoint just returns 200 unconditionally, which tells you nothing useful.

Custom Health Check

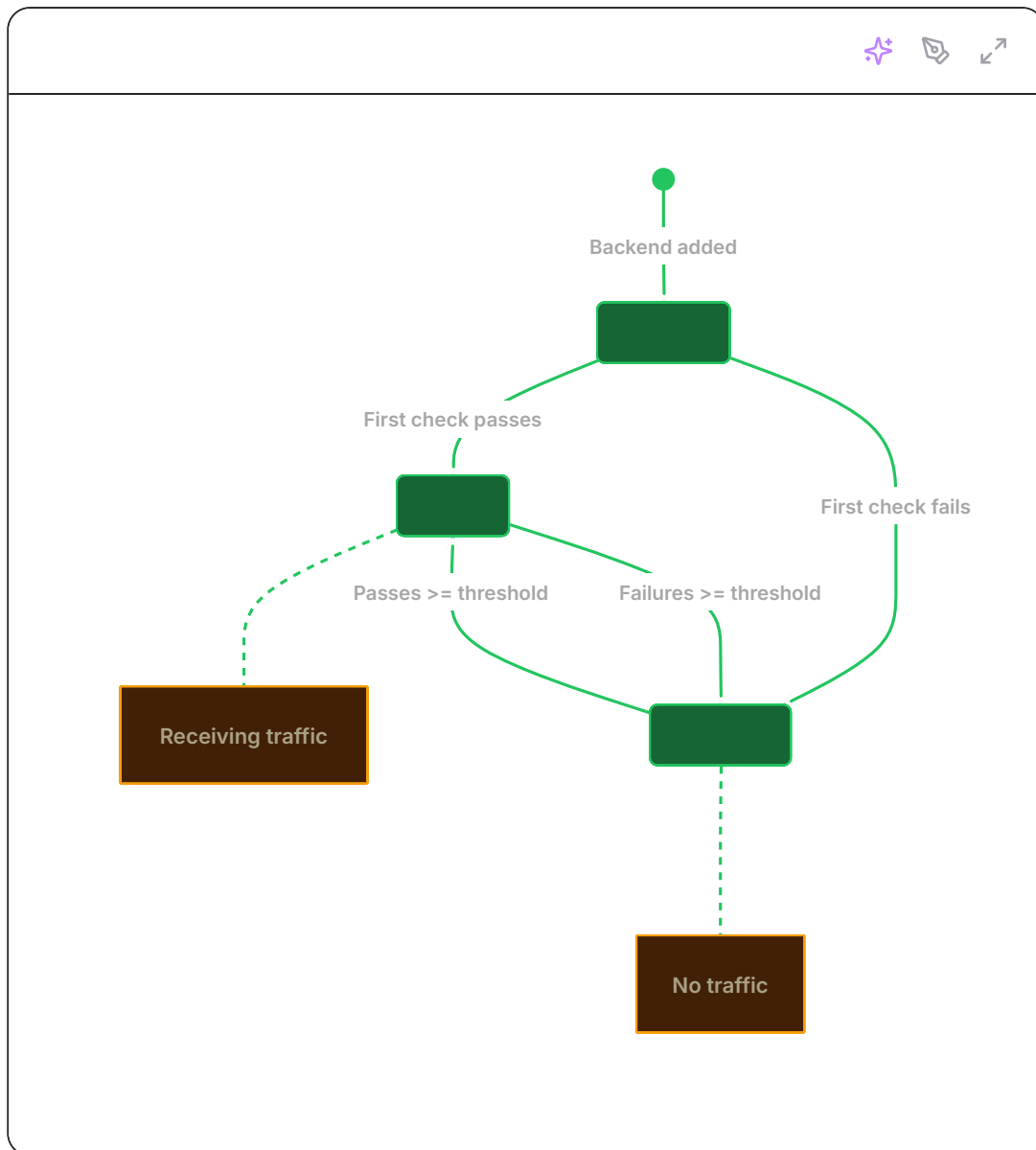
For complex scenarios, you can run a custom script that returns success or failure. This is useful when:

- You need to check something the load balancer cannot access directly (internal databases)

- Health determination requires complex logic
- You are integrating with legacy systems

The Health State Machine

Backends transition through states based on health check results. Understanding these transitions helps you tune the behavior.

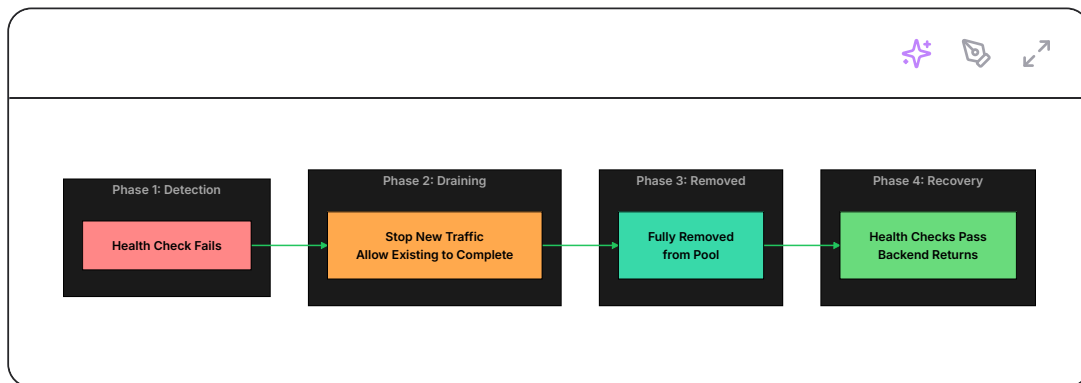


The thresholds prevent flapping. If a backend fails one check (maybe a brief network blip), it does not immediately get marked unhealthy. Only after 2-3 consecutive failures does the status change. Similarly, a backend that was down needs to pass multiple checks before being trusted with traffic again.

Graceful Degradation: Connection Draining

When a backend fails health checks, you should not just immediately drop all connections to it. That would cause errors for requests that are mid-flight. Instead, use connection draining:

1. **Stop new requests:** The routing engine stops sending new requests to the failing backend immediately.
2. **Allow existing connections to complete:** Requests that are already being processed are allowed to finish (up to a timeout, e.g., 30 seconds).
3. **Fully remove:** Once all existing connections complete (or timeout), the backend is fully removed from the pool.



This graceful degradation ensures users experience minimal disruption when backends fail.

6.3 Session Persistence (Sticky Sessions)

Not all applications are stateless. Many legacy systems (and some modern ones) store user session data in local memory on the backend server. If User A logs in on Backend 1, their session lives in Backend 1's memory. If their next request goes to Backend 2, they appear logged out.

Session persistence (also called "sticky sessions") solves this by ensuring that requests from the same user consistently go to the same backend.

The Problem

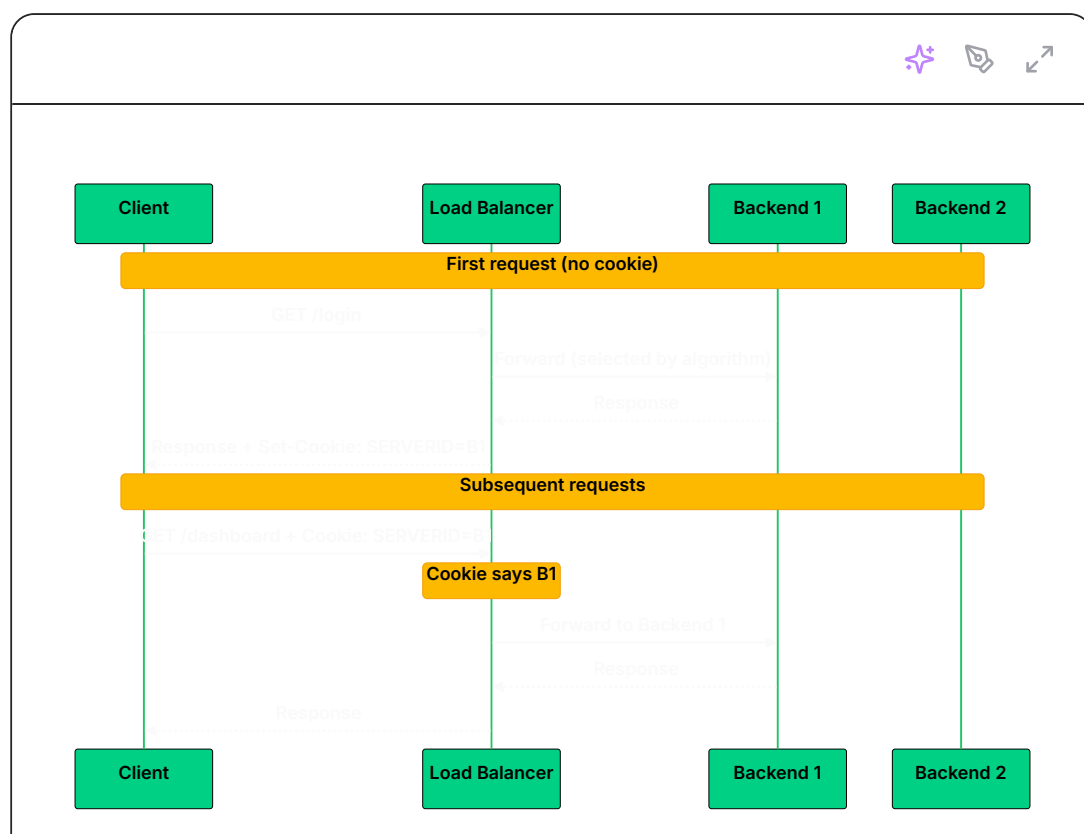
Without stickiness, the load balancer distributes requests without regard to who is making them:

```
Request 1 from User A → Backend 1 (user logs in, session created)
Request 2 from User A → Backend 2 (session not found, user session expires)
Request 3 from User A → Backend 1 (session found, user is logged in)
```

This creates a confusing, broken experience. Session persistence fixes it.

Approach 1: Cookie-Based Stickiness

The most common and reliable approach for HTTP traffic. The load balancer injects a cookie that identifies which backend should handle the user's requests.



How it works:

1. User's first request has no sticky cookie. The load balancer picks a backend using the normal algorithm.
2. The response includes a `Set-Cookie` header with the backend identifier (e.g., `SERVERID=backend-1`).
3. The browser includes this cookie in all subsequent requests.
4. The load balancer reads the cookie and routes directly to the specified backend.

Why this is the best option for HTTP:

- Works reliably regardless of client IP changes (mobile users, corporate proxies)
- No shared state needed between load balancer nodes
- Cookie survives load balancer restarts
- Can include a signature to prevent tampering

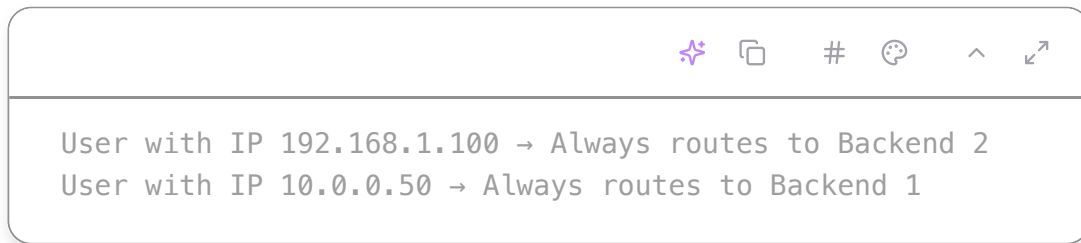
Limitations:

- Requires Layer 7 (HTTP) inspection, does not work for raw TCP traffic
- If the specified backend is unhealthy, the load balancer must pick a new one (and update the cookie)
- Adds a small amount of overhead to read and potentially set cookies

Approach 2: Source IP Persistence

For non-HTTP traffic or when you cannot use cookies, you can route based on client IP address. This is essentially the IP Hash algorithm applied for persistence.

How it works:



When it works:

- Each client has a unique, stable public IP
- You need persistence for non-HTTP protocols
- You cannot modify the application or client

When it breaks:

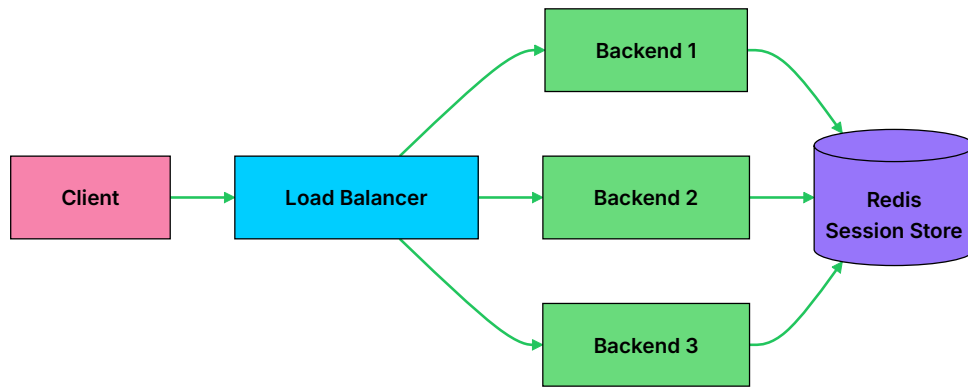
- Corporate NAT: Thousands of users behind one IP all go to the same backend
- Mobile users: IP changes as they move between networks
- IPv4 exhaustion: More clients share fewer IPs

Use IP-based persistence only when cookie-based is not an option.

Approach 3: Externalized Session Store

The cleanest solution is to avoid the problem entirely by making your application stateless. Instead of storing session data in backend memory, store it in a shared session store like Redis.





How it works:

1. User logs in on Backend 1. Backend 1 creates a session and stores it in Redis.
2. User's next request goes to Backend 2. Backend 2 looks up the session in Redis and finds it.
3. Any backend can handle any request since they all share the same session data.

Why this is the best long-term solution:

- No sticky sessions needed, the load balancer can use pure round robin or least connections
- Backends can scale horizontally without concern for session affinity
- Backend failures do not cause session loss (session is safe in Redis)
- Simpler load balancer configuration

The trade-off:

This requires application changes. You need to configure your framework to use Redis (or another external store) instead of local memory for sessions. For new applications, this is easy. For legacy applications, it might require significant refactoring.

Recommendation

If you are building a new application, design it to be stateless from the start. Use an external session store (Redis is the go-to choice) and avoid the complexity of sticky sessions entirely.

For legacy applications that cannot be modified, use cookie-based stickiness for HTTP traffic. It is the most reliable option.

Avoid IP-based persistence unless you have no other choice.

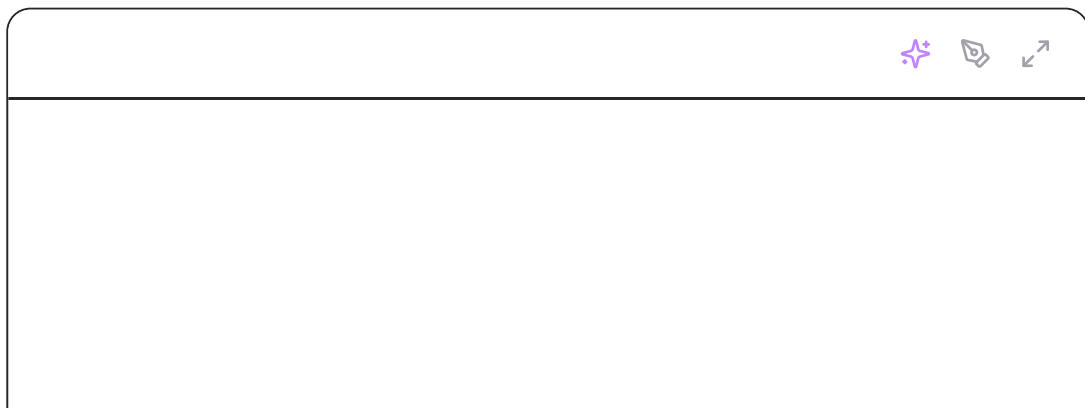
6.4 Layer 4 vs Layer 7 Load Balancing

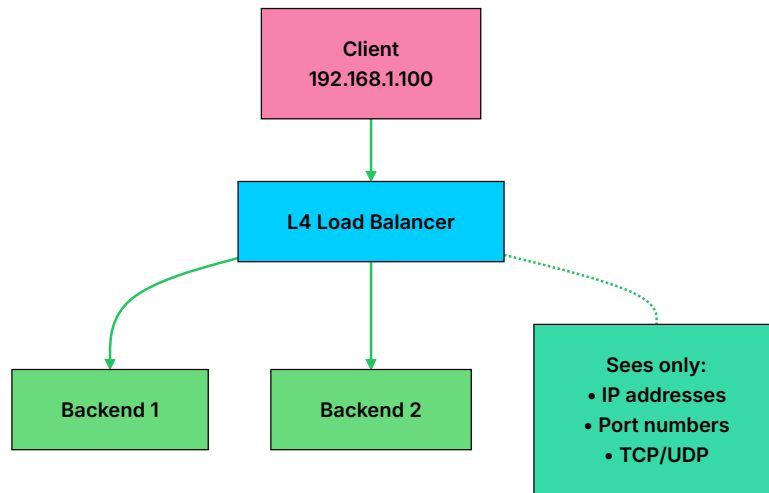
When discussing load balancers, you will often hear terms like "L4" and "L7." These refer to different layers of the network stack where the load balancer makes its decisions. The choice between them affects what the load balancer can do and how fast it can do it.

Layer 4 Load Balancing

A Layer 4 load balancer operates at the transport layer. It sees TCP/UDP packets but does not understand what is inside them. It makes routing decisions based only on:

- Source and destination IP addresses
- Source and destination ports
- Protocol (TCP or UDP)





What L4 can do:

- Route based on port (send traffic on port 80 to web servers, port 3306 to database servers)
- Session persistence based on client IP
- Health checks (TCP connection success/failure)

What L4 cannot do:

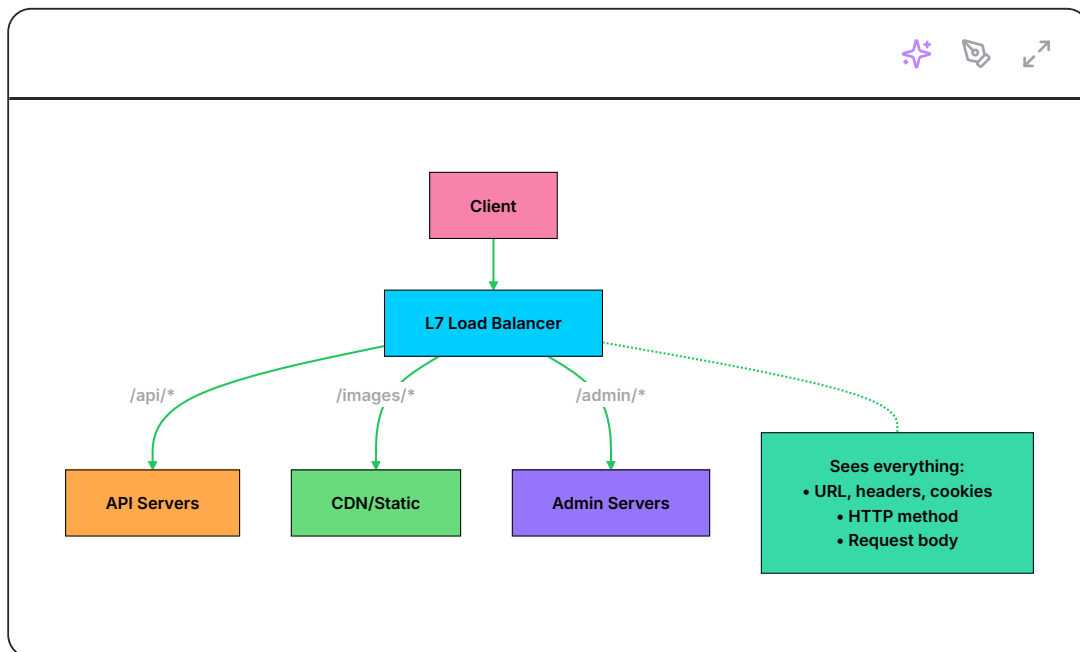
- Route based on URL path (it does not understand HTTP)
- Set or read cookies
- Terminate SSL (it cannot decrypt the traffic)
- Modify headers or content

Why use L4?

Speed. Since L4 does not parse application protocols, it is fast. Some hardware L4 load balancers can handle tens of millions of packets per second. It is also protocol-agnostic, meaning it works with any TCP/UDP traffic (databases, game servers, custom protocols).

Layer 7 Load Balancing

A Layer 7 load balancer operates at the application layer. It understands HTTP (and sometimes other protocols) and can make intelligent routing decisions based on request content.



What L7 can do:

- Route based on URL path (`/api` goes to API servers, `/static` goes to CDN)
- Route based on hostname (`api.example.com` vs `www.example.com`)
- Route based on HTTP method (GET vs POST)
- Terminate SSL and inspect encrypted traffic
- Set and read cookies for sticky sessions
- Add, modify, or remove HTTP headers
- Compress responses
- Cache content

The trade-off:

L7 is more resource-intensive. The load balancer must parse HTTP, which takes CPU cycles. SSL termination requires even

more processing. A server that could handle 1M packets/second at L4 might handle only 100K requests/second at L7 (rough estimate, varies widely by configuration).

Comparison

Capability	Layer 4	Layer 7
Speed	Fastest (packet forwarding)	Slower (protocol parsing)
Intelligence	Basic (IP, port)	Rich (URL, headers, cookies)
SSL termination	Pass-through only	Full termination and inspection
Sticky sessions	IP-based only	Cookie, header, or URL-based
Content routing	Not possible	Based on URL, host, method, etc.
Header modification	Not possible	Add, remove, or modify headers
Caching	Not possible	Can cache responses
Protocol support	Any TCP/UDP	HTTP/HTTPS (usually)

When to Use Each

Use Layer 4 when:

- You need maximum performance
- You are load balancing non-HTTP traffic (databases, game servers, custom protocols)
- Simple round-robin distribution is sufficient
- You want to pass SSL through to backends

Use Layer 7 when:

- You need content-based routing (URL paths, hostnames)
- You want SSL termination at the load balancer
- You need cookie-based sticky sessions

- You want to add headers (like `X-Forwarded-For` or `X-Request-ID`)
- You want to cache or compress responses

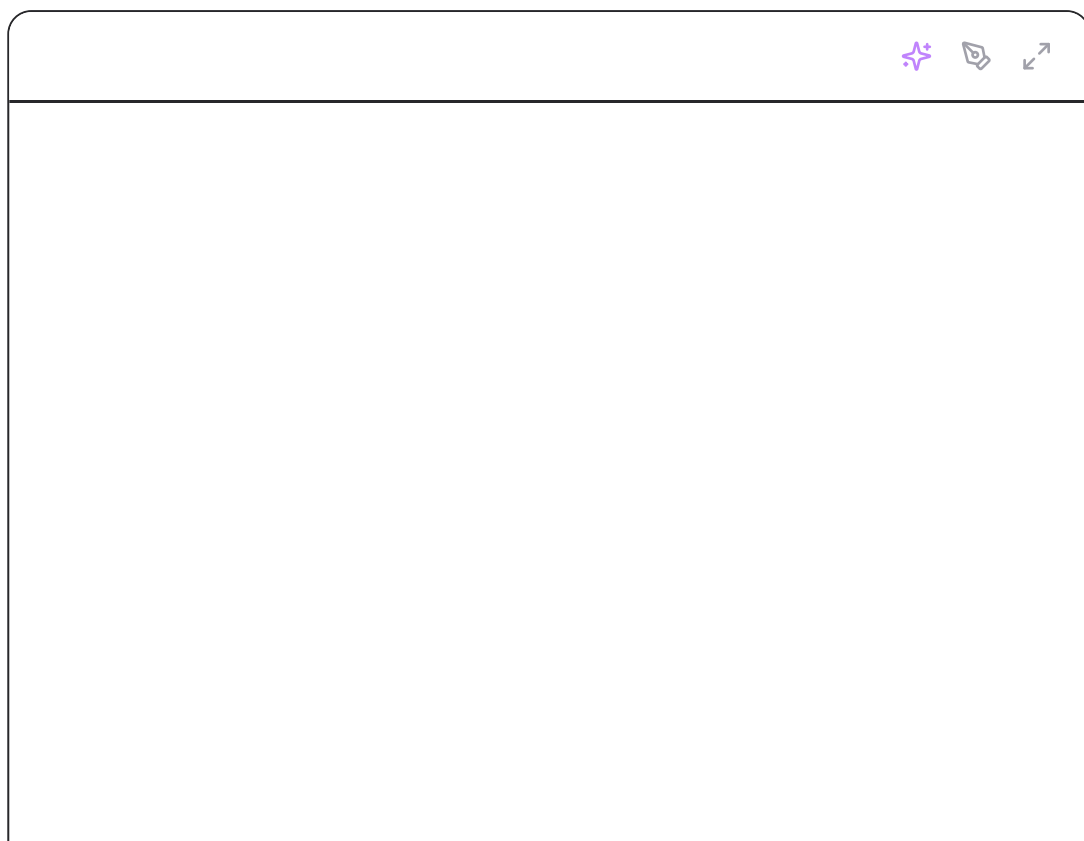
Most modern web applications use L7 load balancers because the additional capabilities outweigh the performance cost. The exception is when you have a layer of L7 load balancers behind an L4 load balancer (the L4 distributes across L7 instances).

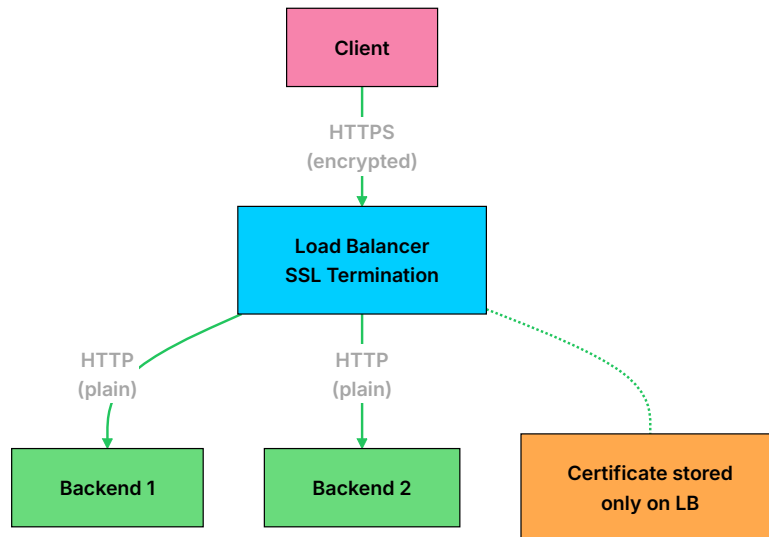
6.5 SSL/TLS Termination

Managing SSL certificates on every backend server is tedious and error-prone. You have dozens of servers, and each needs the certificate renewed before it expires.

This is where SSL termination helps: the load balancer handles all the encryption, and backends receive plain HTTP.

How SSL Termination Works





The client connects to the load balancer over HTTPS. The load balancer decrypts the request, inspects it (for routing decisions), then forwards it to a backend over plain HTTP. The response follows the reverse path.

Why Terminate SSL at the Load Balancer?

Simplified certificate management

You install and renew certificates in one place (the load balancer) instead of on every backend server. When a certificate expires, you update it once, not fifty times.

Offload CPU from backends

SSL encryption and decryption are CPU-intensive operations. By terminating SSL at the load balancer, your backends can use their CPU for application logic instead of cryptography. This is especially valuable if your backends are handling many small requests.

Enable content-based routing

If you want to route based on URL path or headers, the load balancer needs to read the HTTP request. It cannot do this if the traffic is encrypted end-to-end. SSL termination gives the load balancer visibility into the request content.

TLS session caching

The load balancer can cache TLS sessions, so clients reconnecting do not need to do a full TLS handshake every time. This improves latency for repeat visitors.

Security Implications

There is a trade-off: traffic between the load balancer and backends is unencrypted. This is fine if:

- The load balancer and backends are in the same private network (e.g., a VPC)
- You trust the network infrastructure
- You have network-level security controls in place

If you need encryption all the way to the backend (perhaps for compliance reasons), you have options:

Re-encryption

The load balancer decrypts client traffic, inspects it, then re-encrypts it before sending to the backend. This gives you content inspection plus backend encryption, but with double the cryptographic overhead.

SSL Passthrough

The load balancer does not decrypt traffic at all. It routes based on the TLS Server Name Indication (SNI) and forwards encrypted packets directly to backends. You lose the ability to do con-

tent-based routing, but traffic is encrypted end-to-end.

Best Practices

If you are terminating SSL at the load balancer, follow these security practices:

- **Use TLS 1.2 or higher.** Disable older protocols (SSL 3.0, TLS 1.0, TLS 1.1) which have known vulnerabilities.
- **Choose strong cipher suites.** Prefer ECDHE for key exchange (provides forward secrecy) and AES-GCM for encryption.
- **Enable HTTP/2.** Modern browsers support it, and it multiplexes requests over a single connection, improving performance.
- **Use OCSP stapling.** The load balancer fetches certificate validity proofs and includes them in the TLS handshake, so clients do not need to make separate OCSP queries.
- **Automate certificate renewal.** Use tools like Let's Encrypt with automatic renewal so you never have an expired certificate.

6.6 Handling Load Balancer Failures

We covered high availability at a conceptual level in section 4.3. Now let's go deeper into the failure scenarios you need to handle and the mechanisms that make failover work.

The load balancer sits on the critical path for all traffic. If it fails and you do not have a backup, your entire service goes offline. This is why high availability is non-negotiable for production load balancers.

How Failures Are Detected

Before you can recover from a failure, you need to detect it.

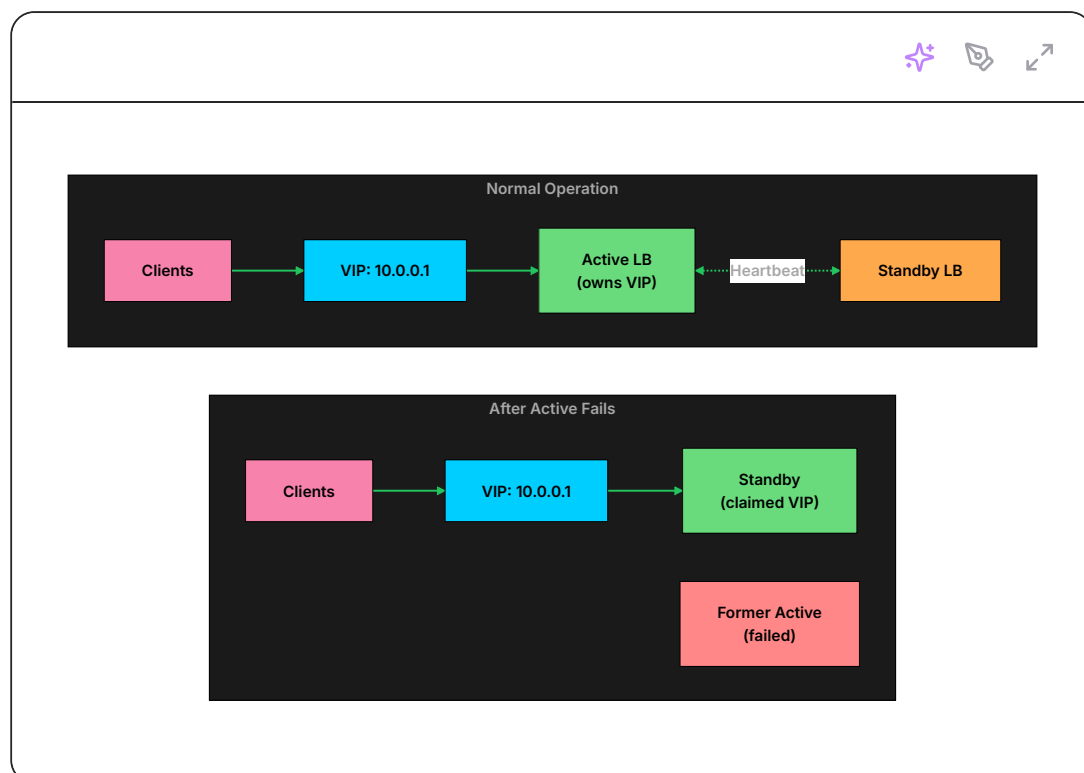
Load balancer nodes monitor each other using:

- **Heartbeat messages:** LB nodes exchange "I'm alive" messages every 1-3 seconds. If a node stops responding, it is presumed dead.
- **Health checks:** The same health checking mechanism used for backends can monitor LB nodes.
- **Shared storage heartbeats:** Write timestamps to a shared location (etcd, Redis) to detect liveness.

The detection threshold is a balance: too sensitive and you get false positives (declaring a node dead when it just had a brief network hiccup), too slow and you extend your outage.

Failover Strategy 1: VRRP (Active-Passive)

VRRP (Virtual Router Redundancy Protocol) is the industry standard for IP failover within a single network segment. Two load balancers share a Virtual IP address, but only one responds to traffic at a time.



How VRRP failover works:

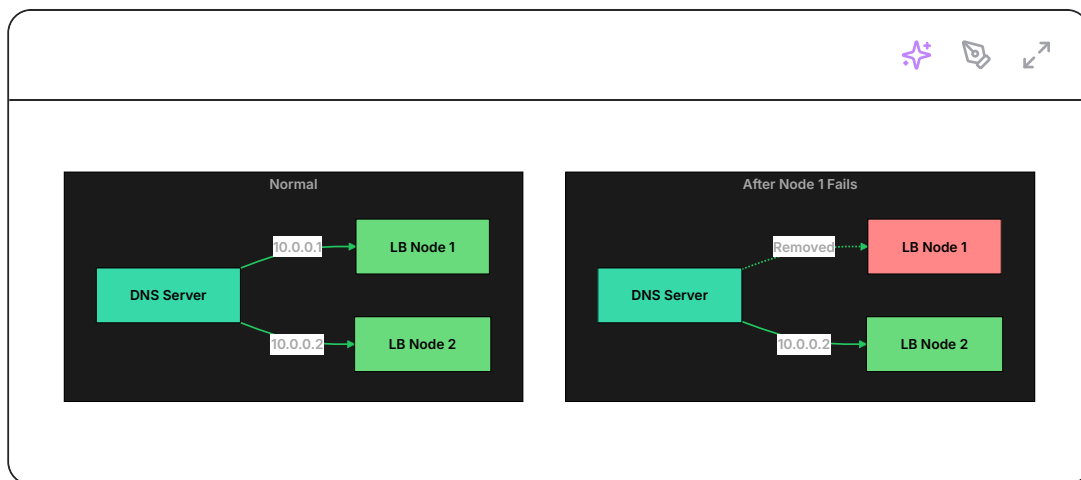
1. Both load balancers run VRRP. The active one (higher priority) "owns" the VIP and responds to ARP requests for it.
2. The standby sends heartbeats to the active every second. As long as heartbeats are acknowledged, the standby stays passive.
3. If the standby misses 3 consecutive heartbeats, it assumes the active is dead. It broadcasts a gratuitous ARP saying "I am now 10.0.0.1" and starts accepting traffic.

Failover time: 1-3 seconds

Trade-off: Simple and reliable, but half your capacity sits idle during normal operation.

Failover Strategy 2: DNS-Based

Multiple load balancers have their own IPs. DNS health checks remove failed ones from rotation.

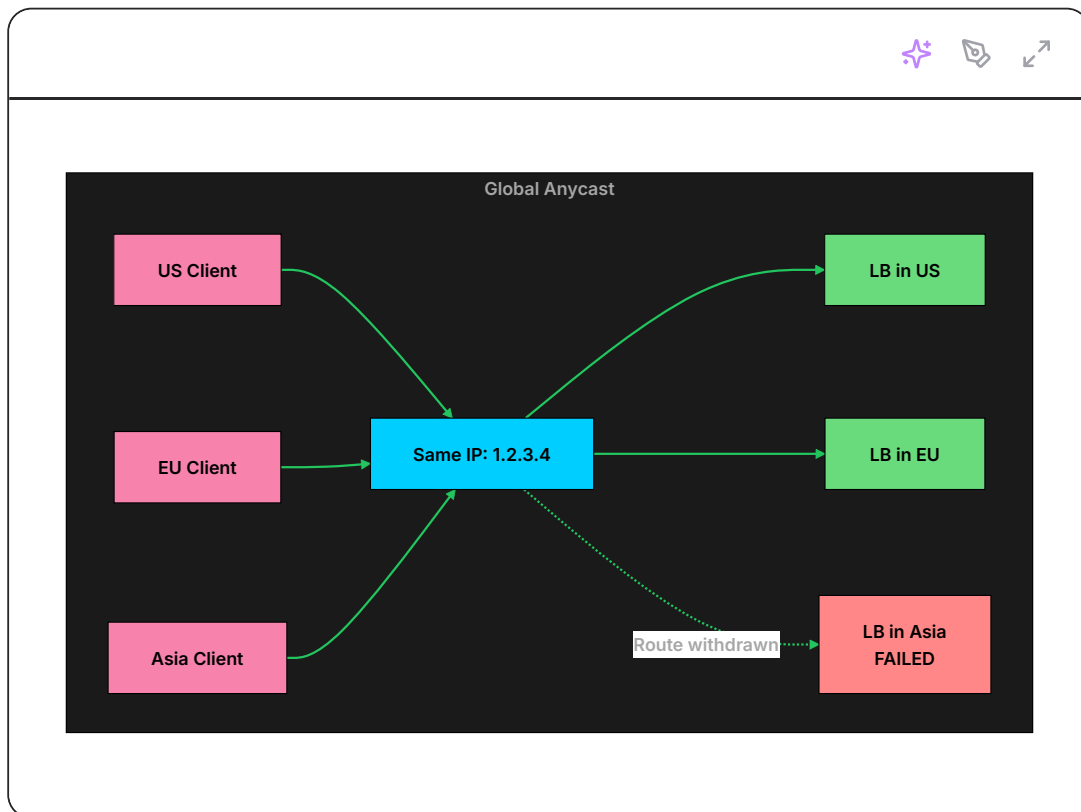


Failover time: Depends on DNS TTL. With a 60-second TTL, failover can take 1-2 minutes as cached DNS entries expire.

Trade-off: Works across data centers (unlike VRRP), but failover is slower.

Failover Strategy 3: Anycast

Multiple load balancers across the world share the same IP address. BGP routing automatically directs traffic to the nearest healthy one.



When the Asia LB fails, it stops announcing its BGP route. Traffic from Asia clients automatically routes to the next nearest healthy LB (US or EU).

Failover time: Seconds (depends on BGP convergence)

Trade-off: Requires BGP expertise and coordination with network providers. Used by Cloudflare, Google, and other global-scale services.

What Happens to Connections During Failover?

When a load balancer fails, connections to it are lost. There is no practical way to transfer an open TCP connection from one machine to another. Clients will see connection resets.

You can minimize the impact:

Strategy	Benefit
----------	---------

Fast failover (VRRP/Any-cast)	Reduces the window where new connections fail
Client retry logic	Applications reconnect automatically
External session store	Users do not lose session data
Short connection timeouts	Clients detect failure faster

Recommendation: Design for stateless failover. Assume connections will be dropped on failure, and build applications that handle retries gracefully. The complexity of stateful failover (synchronizing connection state across LB nodes) is rarely worth it.

References

- [NGINX Load Balancing Documentation](#) - Comprehensive guide on HTTP load balancing with NGINX
- [HAProxy Documentation](#) - Detailed documentation for HAProxy configuration and algorithms
- [AWS Elastic Load Balancing](#) - AWS documentation covering ALB, NLB, and CLB
- [Google Cloud Load Balancing](#) - Google's approach to global load balancing

Quiz



Design Load Balancer Quiz

20 quizzes

Start >

How helpful was this content?

1

2

3

4

5

6

7

8

9

10

Comments (1)

Add a comment...

H1H2BIBU<>📄🔗☰☷🔗0/2000

Comment

Search comments...

Sort byTop

GT Guru Tech 5 months ago



very good Doc -thanks

